

CONTROLADOR LÓGICO PROGRAMÁVEL

D.Sc. William da Silva Vianna

**INSTITUTO FEDERAL FLUMINENSE
DE EDUCAÇÃO CIÊNCIA E TECNOLOGIA**

**CAMPOS DOS GOYTACAZES – RJ
OUTUBRO – 2023**

Sumário

1 INTRODUÇÃO.....	6
2 INFORMAÇÕES GERAIS.....	6
2.1 CARACTERÍSTICAS.....	6
2.2 BREVE HISTÓRICO.....	7
2.3 EVOLUÇÃO.....	7
2.4 APLICAÇÕES.....	8
3 ESTRUTURA BÁSICA.....	9
3.1 MICROPROCESSADOR.....	10
3.1.1 <i>Processamento Cíclico</i>	10
3.1.2 <i>Processamento por interrupção</i>	11
3.1.3 <i>Processamento comandado por tempo</i>	11
3.1.4 <i>Processamento por evento</i>	11
3.2 MEMÓRIA.....	12
3.2.1 <i>Mapa de memória</i>	12
3.2.2 <i>Arquitetura de memória de um CLP</i>	13
3.2.3 <i>Estrutura do mapa de memória do CLP</i>	14
3.3 INTERFACE DE ENTRADA E SAÍDA DO CLP.....	16
3.3.1 <i>Características das entradas e saídas - e/s</i>	17
3.3.2 <i>Terminal de programação</i>	27
4 PRINCÍPIO DE FUNCIONAMENTO DE UM CLP.....	29
4.1 ESTADOS DE OPERAÇÃO.....	29
4.2 FUNCIONAMENTO.....	29
5 LINGUAGEM DE PROGRAMAÇÃO.....	34
5.1 CLASSIFICAÇÃO DAS LINGUAGENS CODE.....	34
5.1.1 <i>Linguagem de baixo nível</i>	34
5.1.2 <i>Linguagem de alto nível</i>	35
6 PROGRAMAÇÃO DE CONTROLADORES PROGRAMÁVEIS.....	36
6.1 <i>LADDER DIAGRAM (LD) - DIAGRAMA DE CONTATOS</i>	37
6.2 <i>FUNCTION BLOCKS DIAGRAM (FBD) - DIAGRAMA DE BLOCOS</i>	37
6.3 <i>INSTRUCTION LIST (IL) - LISTA DE INSTRUÇÃO</i>	38
6.4 <i>STRUCTURED TEXT (ST) – TEXTO ESTRUTURADO</i>	38
6.5 <i>SEQUENTIAL FUNCTION CHART (SFC) - PASSOS OU STEP</i>	39
6.6 <i>ANÁLISE DAS LINGUAGENS DE PROGRAMAÇÃO</i>	39
6.7 <i>NORMALIZAÇÃO - IEC 61131</i>	41

6.7.1 Elementos Comuns.....	43
6.7.2 Linguagens da norma IEC 61131-3.....	43
7 PROGRAMAÇÃO EM LADDER.....	47
7.1 DESENVOLVIMENTO DO PROGRAMA LADDER.....	49
7.1.1 Associação de contatos no ladder.....	52
7.1.2 Instruções.....	54
7.1.3 Instruções básicas.....	55
8 NOÇÕES DE SISTEMA SCADA COM USO DO CLP.....	67
8.1 ARQUITETURA DA REDE CLP PARA SISTEMAS SCADA.....	68
9 CRITÉRIOS PARA AQUISIÇÃO DE UM CLP.....	70
9.1 CRITÉRIOS DE CLASSIFICAÇÃO.....	70
9.2 CRITÉRIOS DE AVALIAÇÃO PARA ESPECIFICAÇÃO E COMPRA DE UM CP.....	71
9.3 ANÁLISE DO FORNECEDOR.....	71
9.4 ASPECTOS TÉCNICOS DO PRODUTO.....	72
9.5 ASPECTOS CONTRATUAIS.....	72
10 BIBLIOGRAFIA RECOMENDADA.....	72

Lista de figuras

Figura 1 - Diagrama em blocos com os componentes básicos de um CLP.....	9
Figura 2 - Esquema do processamento cíclico.....	10
Figura 3 - Esquema do processamento por interrupção.....	11
Figura 4 - Esquema do mapa de memória.....	12
Figura 5 - Esquema da divisão dos tipos de memória primária.....	13
Figura 6 - Esquema da arquitetura de um CLP com interfaces de entrada e saída.....	17
Figura 7 - Esquema do cartão ou módulo de entrada digital com respectivos elementos de campo.....	18
Figura 8 - Circuito de entrada digital opto isolado.....	19
Figura 9 - Esquema do circuito elétrico de ligação de duas entradas digitais com módulo ou cartão entrada em tensão.....	19
Figura 10 - Esquema do circuito elétrico de ligação de uma entradas digitais com módulo ou cartão entrada em tensão e sensor discreto indutivo.....	20
Figura 11 - Esquema do cartão ou módulo de entrada analógica com respectivos elementos de campo.....	20
Figura 12 - Esquema do circuito elétrico de ligação de duas entradas analógicas em corrente com dois transmissores a dois fios (two wire).....	21
Figura 13 - Esquema do circuito elétrico de ligação de duas entradas analógicas em tensão com dois transmissores a dois fios (two wire).....	21
Figura 14 - Diagrama em blocos dos elementos de tratamento do sinal de entrada.....	22
Figura 15 - Diagrama unifilar do cartão ou módulo de saída digital com exemplos de respectivos elementos de campo.....	23
Figura 16 – Esquema do circuito interno e externo de ligação de um ponto de saída digital a transistor.....	23
Figura 17 - Esquema do circuito interno e externo de ligação de um ponto de saída digital a triac.....	24
Figura 18 - Esquema do circuito interno e externo de ligação de um ponto de saída digital a rele ou contato seco.....	24
Figura 19 - Esquema do circuito elétrico de ligação de duas saídas digitais independentes.....	24
Figura 20 - Esquema do circuito elétrico de ligação de duas saídas digitais com um terminal em comum.....	25
Figura 21 - Diagrama unifilar do cartão ou módulo de saída analógica com respectivos exemplos de elementos de campo.....	25
Figura 22 - Esquema do circuito elétrico de ligação de duas saídas analógicas em corrente.....	26
Figura 23 - Exemplo de terminal de programação dedicado portátil ou handheld utilizados em CLPs antigos (legado).....	28
Figura 24 - Exemplo de terminal de programação dedicado TRC (esquerda) utilizado em CLPs antigos (direita).....	28
Figura 25 - Fluxograma de funcionamento do ciclo de operação de um CLP.....	31
Figura 26 - Ilustração do funcionamento da atualização da memória imagem de E/S com utilização da linguagem Ladder.....	33
Figura 27 - Exemplo de programa em Ladder.....	37
Figura 28 - Exemplo de programa em blocos.....	38
Figura 29 - Exemplo de programa em IL.....	38
Figura 30 - Exemplo de programa em ST.....	38
Figura 31 - Exemplo de programa em passos.....	39
Figura 32 - Exemplos básicos de programas escritos utilizando as linguagens da norma IEC 61131-3.....	42
Figura 33 - Exemplo de programa básico SFC.....	44
Figura 34 - Exemplo de um mesmo código implementado nas 4 linguagens IEC.....	45
Figura 35 - Modelo de software proposto pelo padrão IEC 61131-3.....	46
Figura 36 Elementos básicos binários da programação em Ladder.....	47
Figura 37 - Passos para elaboração do programa que irá controlar um processo ou equipamento.....	50
Figura 38 - Circuito básico de acionamento.....	51
Figura 39 - Circuito básico de acionamento em Ladder.....	51
Figura 40 - Linha Ladder com contato inversor.....	51
Figura 41 - Ladder utilizando contato normalmente aberto (A) e Ladder utilizando contato normalmente fechado (B).....	52
Figura 42 - Programa Ladder com lógica E.....	52

Figura 43 - Programa em texto estruturado com lógica E.....	53
Figura 44 - Programa Ladder com lógica E.....	53
Figura 45 - Programa em texto estruturado com lógica OU.....	53
Figura 46 - Programa Ladder com lógica mista.....	54
Figura 47 - Programa em texto estruturado com lógica E.....	54
Figura 48 - Esquema do uso de um bloco funcional no programa Ladder.....	55
Figura 49 - Programa Ladder com bloco funcional de temporização.....	56
Figura 50 - Programa Ladder com bloco funcional de contagem.....	56
Figura 51 - Gráficos para demonstração do funcionamento do contador.....	57
Figura 52 - Programa Ladder com bloco funcional para mover dado.....	57
Figura 53 - Gráficos para demonstração do funcionamento do bloco mover.....	58
Figura 54 - Programa Ladder com blocos funcionais de comparação maior e menor.....	59
Figura 55 - Gráficos para demonstração do funcionamento do bloco de comparação maior e menor.....	59
Figura 56 - Programa Ladder com bloco funcional matemático de adição.....	60
Figura 57 - Gráficos para demonstração do funcionamento do bloco soma.....	61
Figura 58 - Programa Ladder com bloco funcional matemático de subtração.....	61
Figura 59 - Gráficos para demonstração do funcionamento do bloco de subtração.....	62
Figura 60 - Programa Ladder com bloco funcional matemático de multiplicação.....	62
Figura 61 - Programa Ladder com bloco funcional matemático de divisão.....	63
Figura 62 - Programa Ladder com bloco funcional lógica E.....	64
Figura 63 - Programa Ladder com bloco funcional lógica OU.....	65
Figura 64 - Programa Ladder com bloco funcional lógica ou exclusivo.....	66
Figura 65 - Esquema básico de um sistema SCADA.....	68
Figura 66 – Arquitetura local de rede CLP com uso do CLP modular ou compacto.....	69
Figura 67 - Arquitetura local de CLP com I/O remotos ou distribuídos.....	69
Figura 68 - Arquitetura de rede de CLP`s.....	70

1 INTRODUÇÃO

O Controlador Lógico Programável, ou simplesmente CLP, tem revolucionado os comandos e controles industriais desde seu surgimento na década de 70.

Antes do surgimento dos CLPs as tarefas de comando e controle de máquinas e processos industriais eram feitas por relés eletromagnéticos, especialmente projetados para este fim.

O primeiro CLP surgiu na indústria automobilística, até então um usuário em potencial dos relés eletromagnéticos utilizados para controlar operações sequenciadas e repetitivas numa linha de montagem. A primeira geração de CLPs utilizou componentes discretos como transistores e CIs com baixa escala de integração.

Este equipamento foi batizado nos Estados Unidos como PLC (*Programmable Logic Control*), em português CLP (*Controlador Lógico Programável*) e este termo é registrado pela *Allen Bradley* (fabricante de CLPs).

Definição segundo a ABNT (Associação Brasileira de Normas Técnicas)

“É um equipamento eletrônico digital com hardware e software compatíveis com aplicações industriais.”

Definição segundo a Nema (National Electrical Manufacturers Association)

“Aparelho eletrônico digital que utiliza uma memória programável para o armazenamento interno de instruções para implementações específicas, tais como lógica, sequenciamento, temporização, contagem e aritmética, para controlar, através de módulos de entradas e saídas, vários tipos de máquinas ou processos.”

2 INFORMAÇÕES GERAIS

2.1 CARACTERÍSTICAS

Basicamente, um controlador programável apresenta as seguintes características:

- Hardware e/ou dispositivo de controle de fácil e rápida programação ou reprogramação, com a mínima interrupção da produção;
- Capacidade de operação em ambiente industrial;
- Sinalizadores de estado e módulos tipo plug-in de fácil manutenção e substituição;
- Hardware ocupando espaço reduzido e apresentando baixo consumo de energia;
- Possibilidade de monitoração do estado e operação do processo ou sistema, através da comunicação com computadores constituindo sistemas de supervisão controle e aquisição de dados (SCADA);
- Compatibilidade com diferentes tipos de sinais de entrada e saída;

- Capacidade de alimentar, de forma contínua ou chaveada, cargas que consomem correntes de até 2 A quando utilizado o tipo de saída discreta adequada;
- Hardware de controle que permite a expansão dos diversos tipos de módulos, de acordo com a necessidade;
- Custo de compra e instalação competitivo em relação aos sistemas de controle convencionais;
- Possibilidade de expansão da capacidade de memória;
- Conexão com outros CLPs através de rede de comunicação;
- Possibilidade de integração com redes de chão de fábrica;
- Programação em pelo menos uma linguagem com possibilidade de programação em até cinco linguagens distintas em um mesmo programa conforme norma IEC61131-3;

2.2 BREVE HISTÓRICO

O controlador programável nasceu praticamente dentro da indústria automobilística americana, especificamente na Hydromic Division da General Motors, em 1968, devido a grande dificuldade de se mudar a lógica de controle de painéis de comando a cada mudança na linha de montagem. Estas mudanças implicavam em altos gastos de tempo e dinheiro.

Sob a liderança do engenheiro Richard Morley, foi preparada uma especificação que refletia os sentimentos de muitos usuários de relés, não só da indústria automobilística como de toda a indústria manufatureira.

Nascia assim a indústria de controladores programáveis, hoje com um mercado mundial multibilionário

2.3 EVOLUÇÃO

Desde o seu aparecimento até hoje, muita coisa evolui nos controladores lógicos. Esta evolução está ligada diretamente ao desenvolvimento tecnológico da informática em suas características de software e de hardware.

O que no seu surgimento era executado com componentes discretos, hoje se utiliza de microprocessadores e microcontroladores de última geração, usando técnicas de processamento paralelo, inteligência artificial, redes de comunicação, fieldbus, etc.

Até recentemente não havia nenhuma padronização entre fabricantes, apesar da maioria utilizar as mesmas normas construtivas. Porém, pelo menos ao nível de software aplicativo, os controladores programáveis podem se tornar compatíveis com a adoção da norma IEC 61131-3, que prevê a padronização da linguagem de programação e sua portabilidade.

Outra novidade que foi incorporada aos controladores programáveis é o fieldbus (barramento de chão-de-fábrica), que surge como uma proposta de padronização de sinais a nível de chão-de-fábrica. Este barramento se propõe a diminuir sensivelmente o número de condutores usados para interligar os sistemas de controle aos sensores e atuadores, além de propiciar os ganhos da utilização de equipamentos microprocessados no chão-de-fábrica.

Hoje os CLPs oferecem um considerável número de benefícios para aplicações industriais, que podem ressaltar em economia que excede o custo do CLP e devem

ser considerados quando da seleção de um dispositivo de controle industrial. As vantagens de sua utilização, comparados a outros dispositivos de controle industrial incluem:

- Menor ocupação de espaço;
- Potência elétrica requerida menor;
- Reutilização;
- Reprogramável, se ocorrerem mudanças de requisitos de controle;
- Confiabilidade maior;
- Manutenção mais fácil;
- Maior flexibilidade, satisfazendo um maior número de aplicações;
- Permite a interface através de rede de comunicação com outros CLPs e microcomputadores;
- Projeto do sistema mais rápido.

Todas estas considerações mostram a evolução de tecnologia, tanto de hardware quanto de software, o que permite o seu acesso a um maior número de pessoas tanto nos projetos de aplicação de controladores programáveis quanto na sua programação.

2.4 APLICAÇÕES

O controlador programável existe para automatizar processos industriais, sejam de sequenciamento, intertravamento, controle de processos, batelada, etc.

Este equipamento tem seu uso tanto na área de automação da manufatura, de processos contínuos, elétrica, predial, entre outras.

Praticamente não existem ramos de aplicações industriais onde não se possa aplicar os CLPs, entre elas tem-se:

- Máquinas industriais (operatrizes, injetoras de plástico, têxteis, calçados);
- Equipamentos industriais para processos (siderurgia, papel e celulose, petroquímica, química, alimentação, mineração, etc);
- Equipamentos para controle de energia (demanda, fator de carga);
- Controle de processos com realização de sinalização, intertravamento, controle PID, controle avançado;
- Aquisição de dados de supervisão em: fábricas, prédios inteligentes, etc;
- Bancadas de teste automático de componentes industriais;
- Testes de conformidade em aplicações laboratoriais
- Controle de sistemas embarcados em diversas aplicações aeroespaciais;
- Etc.

3 ESTRUTURA BÁSICA

O controlador programável tem sua estrutura baseada no hardware de um computador, tendo portanto uma unidade central de processamento (CPU), interfaces de entrada e saída e memórias.

As principais diferenças em relação a um computador comum estão relacionadas a qualidade da fonte de alimentação, que possui características ótimas de filtragem e estabilização, interfaces de E/S imune a ruídos e um invólucro específico para aplicações industriais. Tem-se também um terminal usado para programação do CLP. Esse terminal pode ser dedicado ou um microcomputador padrão com software de programação específico para o equipamento que será programado. Atualmente o uso do terminal dedicado para programação está em desuso restando apenas o hardware legado.

O diagrama de blocos a seguir, ilustra a estrutura básica de um controlador programável:

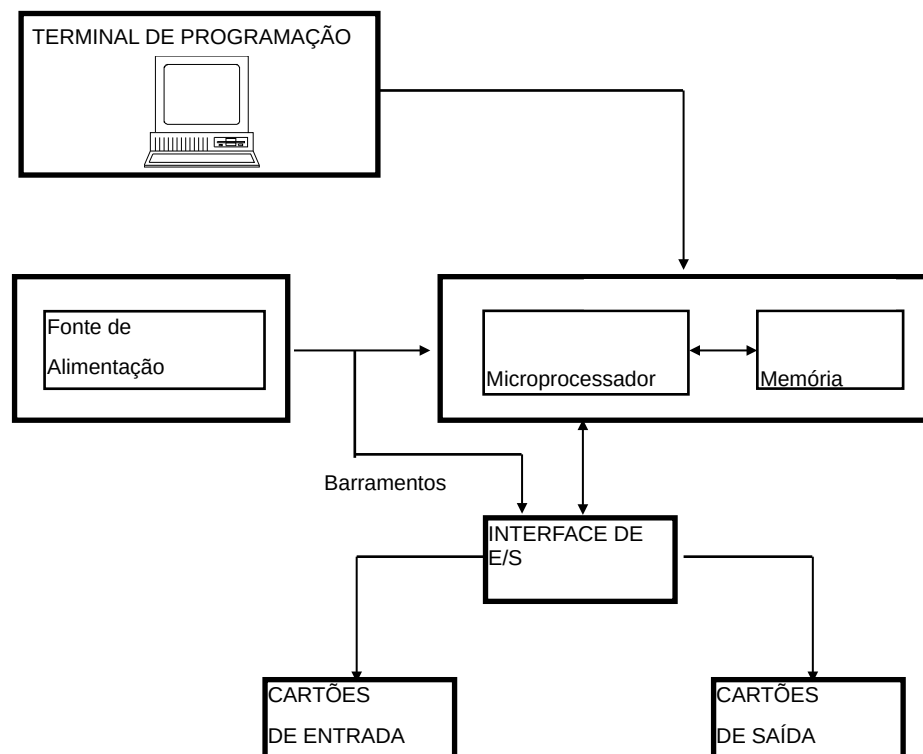


Figura 1 - Diagrama em blocos com os componentes básicos de um CLP.

Dentre as componentes integrantes desta estrutura tem-se:

- Microprocessador;
- Memória;
- Barramentos;
- E/S (Entradas e Saídas);
- Terminal de Programação;
- Fonte de alimentação.

3.1 MICROPROCESSADOR

O microprocessador é responsável pelo processamento do programa, isto é, coleta os dados dos cartões de entrada, efetua o processamento segundo o programa do usuário, armazenado na memória, e envia o dado para os cartões de saída como resposta ao processamento.

Geralmente, cada CLP tem um microprocessador, que pode controlar vários pontos de E/S (entradas e saídas). De acordo com a estrutura física dos pontos de E/S (I/O) o CLP pode ser classificado em:

- **CLP compacto** – quando todos os componentes do CLP são fabricados em um único módulo (exceto terminal de programação). Frequentemente o CLP compacto é empregado em pequenas aplicações. Além disso, geralmente faz-se uso de microcontroladores em substituição ao microprocessador, memória e I/O;
- **CLP modular** – constituído de um ou mais racks que permitem a montagem da fonte de alimentação, Unidade Central de Processamento (CPU) e cartões de entrada e saída;
- **CLP com I/O distribuído** – constituído de pelo menos uma CPU, fonte e diversos módulos de E/S distribuídos e comunicando com a CPU por meio de uma rede fieldbus.

Independente da classificação o processamento poderá ter estruturas diferentes para a execução de um programa, tais como:

- **Processamento cíclico;**
- **Processamento por interrupção;**
- **Processamento comandado por tempo;**
- **Processamento por evento.**

3.1.1 PROCESSAMENTO CÍCLICO

É a forma mais comum de execução que predomina em todas as CPUs conhecidas, e de onde vem o conceito de varredura, ou seja, as instruções de programa contidas na memória, são lidas uma após a outra sequencialmente do início ao fim, daí retornando ao início ciclicamente.

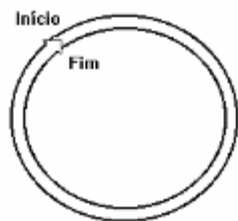


Figura 2 - Esquema do processamento cíclico.

Um dado importante de uma CPU é o seu tempo de ciclo, ou seja, o tempo gasto para a execução de uma varredura. Este tempo está relacionado com o tamanho do programa do usuário e capacidade de processamento. Valores típicos estão na ordem de poucos milissegundos, mas com a evolução da eletrônica embarcada os valores podem atingir ordem de alguns microssegundos.

3.1.2 PROCESSAMENTO POR INTERRUPÇÃO

Certas ocorrências no processo controlado não podem, algumas vezes, aguardar o ciclo completo de execução do programa. Neste caso, ao reconhecer uma ocorrência deste tipo, a CPU interrompe o ciclo normal de programa e executa um outro programa chamado de rotina de interrupção.

Esta interrupção pode ocorrer a qualquer instante da execução do ciclo de programa. Ao finalizar esta situação o programa voltará a ser executado do ponto onde ocorreu a interrupção.

Uma interrupção pode ser necessária, por exemplo, numa situação de emergência onde procedimentos referentes a esta situação devem ser adotados.

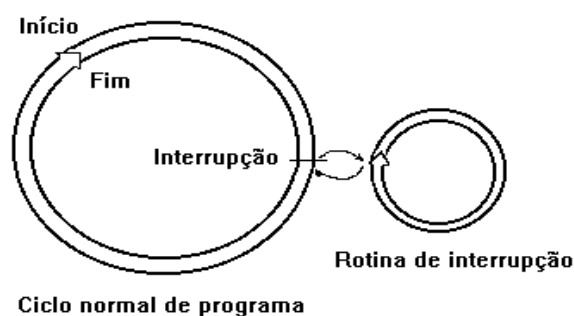


Figura 3 - Esquema do processamento por interrupção

3.1.3 PROCESSAMENTO COMANDADO POR TEMPO

Da mesma forma que determinadas execuções não podem ser dependentes do ciclo normal de programa, algumas devem ser executados a certos intervalos de tempo, às vezes muito curto, na ordem de milissegundos.

Este tipo de processamento também pode ser encarado como um tipo de interrupção, porém ocorre a intervalos regulares de tempo dentro do ciclo normal de programa.

3.1.4 PROCESSAMENTO POR EVENTO

Este é processado em eventos específicos, tais como no retorno de energia, falha na bateria (para CLPs legados) e estouro do tempo de supervisão do ciclo da CPU.

Neste último, tem-se o chamado *Watch Dog Time* (WDT), que normalmente ocorre como procedimento ao se detectar uma condição de estouro de tempo de ciclo da CPU, parando o processamento numa condição de falha e indicando ao operador através de sinal visual e as vezes sonoro.

3.2 MEMÓRIA

O sistema de memória é uma parte de vital importância no processador de um controlador programável, pois armazena todas as instruções assim como os dados necessários para executá-las.

Existem diferentes tipos de sistemas de memória. A escolha de um determinado tipo depende:

- **do tipo de informação armazenada;**
- **da forma como a informação será processada pela CPU.**

As informações armazenadas num sistema de memória são chamadas *palavras de memória*, que são formadas sempre com o mesmo número de bits.

A capacidade de memória de um CP é definida em função do número de palavras de memória previstas para o sistema.

3.2.1 MAPA DE MEMÓRIA

A capacidade de memória de um CP pode ser representada por um mapa chamado *mapa de memória*.

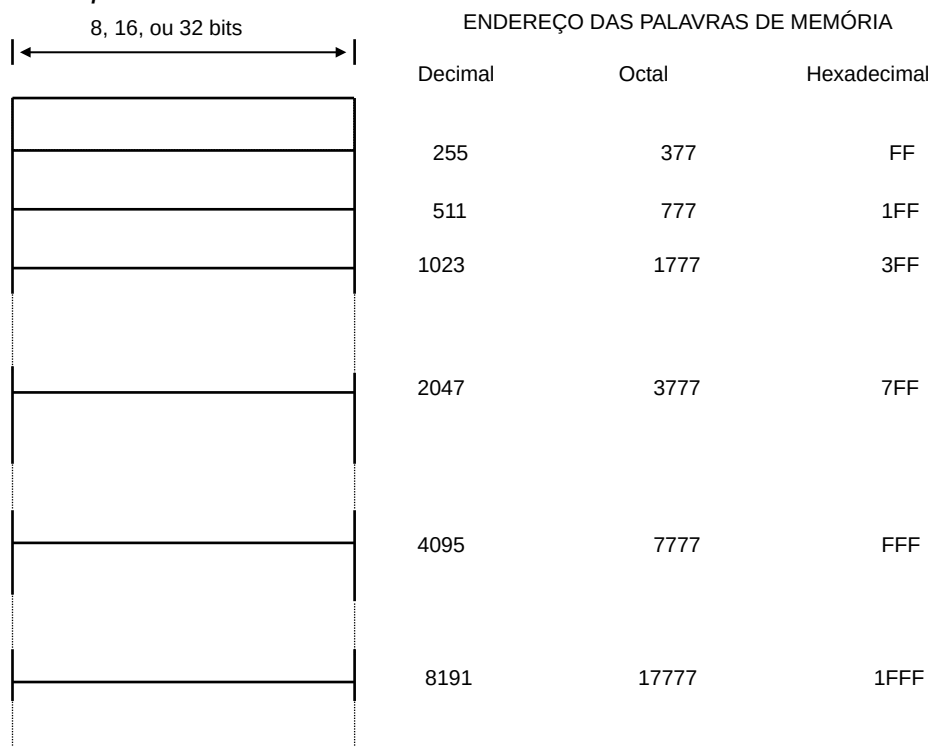


Figura 4 - Esquema do mapa de memória.

O tamanho da palavra de memória dependerá de características como:

- Tipo de processador utilizado;
- Projeto eletrônico dos circuitos CLP.

3.2.2 ARQUITETURA DE MEMÓRIA DE UM CLP

A arquitetura de memória de um controlador lógico programável pode ser constituída por diferentes tipos de memória conforme o uso.

A memória do CLP pode ser classificada conforme o que estará armazenado. De forma geral são utilizadas memórias ROM e RAM.

As memórias a semicondutores podem ser divididas em dois grupos diferentes:

- Memória **ROM** (read only memory) memória apenas de leitura.
- Memória **RAM** (random access memory) memória de acesso aleatório.

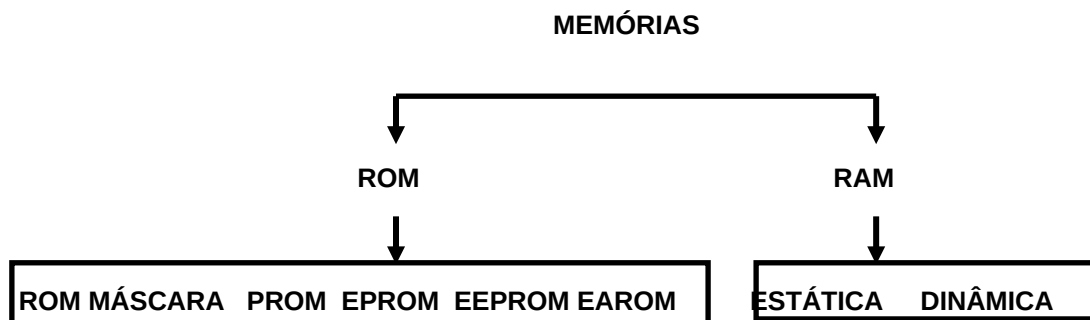


Figura 5 - Esquema da divisão dos tipos de memória primária

As memórias ROM são designadas como memória de programa por serem memórias que não podem ser alteradas em estado normal de funcionamento, porém têm a vantagem de não perderem as suas informações mesmo quando é desligada sua alimentação.

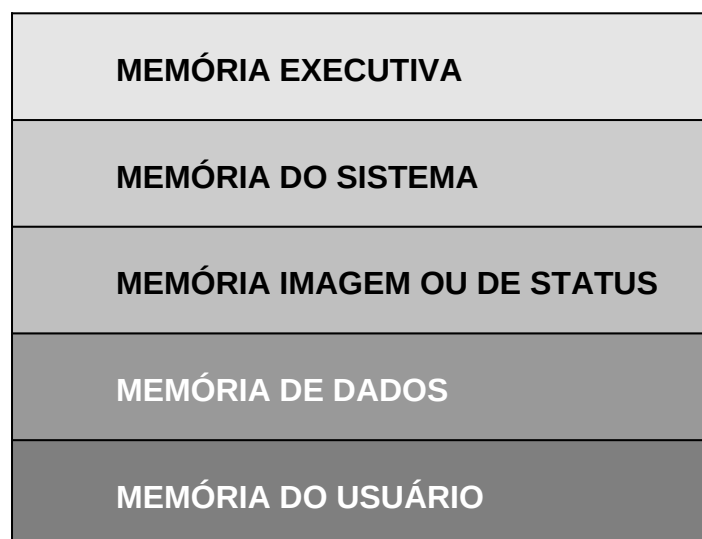
As memórias RAM e ROM são classificadas como memórias primária. Já os discos rígidos, SSDs, eMMC, mídias como cartões de memória e discos flexíveis são classificados como memórias secundárias. Os CLP's não possuem discos rígidos, mas atualmente com o desenvolvimento da eletrônica podem estar disponíveis no mercado CLPs com memórias como eMMC e cartões tipo micro SD.

Tabela 1 - Tipos de memória.

Tipo de Memória	Descrição	Característica
RAM DINÂMICA	Memória de acesso aleatório	- Volátil - Gravada pelo usuário - Lenta - Ocupa pouco espaço - Menor custo
RAM ESTÁTICA	Memória de acesso aleatório	- Volátil - Gravada pelo usuário - Rápida - Ocupa mais espaço - Maior custo
ROM MÁSCARA	Memória somente de leitura	- Não Volátil - Não permite apagamento - Gravada pelo fabricante
PROM	Memória programável somente de leitura	- Não volátil - Não permite apagamento - Gravada pelo usuário
EPROM	Memória programável/apagável somente de leitura	- Não Volátil - Apagamento por ultravioleta - Gravada pelo usuário
EEPROM FLASH EPROM	Memória programável/apagável somente de leitura	- Não Volátil - Apagável eletricamente - Gravada pelo usuário

3.2.3 ESTRUTURA DO MAPA DE MEMÓRIA DO CLP

Independente dos tipos de memórias utilizadas, o mapa de memória de um controlador programável pode ser dividido em cinco áreas principais:



Memória Executiva

É formada por memórias do tipo **ROM** e em seu conteúdo está armazenado o *sistema operacional* responsável por todas as operações que são realizadas no CLP. Este sistema também é conhecido como firmware.

O usuário geralmente não tem acesso a esta área de memória.

Memória do Sistema

Esta área é formada por memórias tipo **RAM**, pois terá o seu conteúdo constantemente alterado pelo sistema operacional também conhecido como firmware do CLP.

Armazena resultados e/ou operações intermediárias, geradas pelo sistema, quando necessário. Pode ser considerada como um tipo de *rascunho*.

Nos CLPs legados não pode ser acessada nem alterada pelo usuário. Em alguns CLPs mais novos é possível realizar intervenções.

Memória de Status de E/S ou Memória Imagem

A memória de status dos módulos de E/S são do tipo **RAM**. A CPU, após ter efetuado a leitura dos estados de todas as entradas, armazena essas informações na área denominada *status das entradas* ou *imagem das entradas*. Após o processamento dessas informações, os resultados serão armazenados na área denominada *status das saídas* ou *imagem das saídas*.

Memória de Dados

As memórias de dados são do tipo **RAM**, e armazenam valores do processamento das instruções utilizadas pelo programa do usuário.

Funções de temporização, contagem, controle PID, aritméticas e especiais, necessitam de uma área de memória para armazenamento de dados, como:

- *valores pré-selecionados ou acumulados de contagem e temporização;*
- *resultados ou variáveis de operações aritméticas;*
- *resultados ou dados diversificados a serem utilizados por funções de manipulação de dados.*

Memória do Usuário

A CPU efetuará a leitura das instruções contidas nesta área a fim de executar o programa do usuário, de acordo com os procedimentos predeterminados pelo sistema operacional.

As memórias destinadas ao usuário podem ser do tipo:

- **RAM**
- **RAM/EPROM**
- **RAM/EEPROM/Flash**

Tabela 2 - Memória RAM do usuário.

Tipo de Memória	Descrição
RAM	A maioria do CLPs utiliza memórias RAM para armazenar o programa do usuário assim como os dados internos do sistema. Nos CLPs legados geralmente está associada a baterias internas que evitarão a perda das informações em caso de queda da alimentação.
RAM/EPROM (legado)	O usuário desenvolve o programa e efetua testes em RAM. Uma vez checado o programa, este é transferido para EPROM.
RAM/EEPROM/Flash	Esta configuração de memória do usuário permite que, uma vez definido o programa, este seja copiado em EEPROM/Flash. Uma vez efetuada a cópia, o CLP poderá operar tanto em RAM como em EEPROM/Flash. Para qualquer modificação bastará um comando via software, e este tipo de memória será apagada e gravada eletricamente.

3.3 INTERFACE DE ENTRADA E SAÍDA DO CLP

As interfaces de entrada e saída do CLP são os circuitos responsáveis pela interação entre o mundo externo e a máquina; são os circuitos por onde os sensores e outros podem introduzir informações na máquina ou por onde a máquina pode enviar informações ao mundo externo. Como interface de entrada pode-se citar os seguintes exemplos: entradas analógicas, entradas digitais, porta serial, entrada digital de contagem rápida, etc. Estas interfaces tem por função a transformação dos sinais elétricos em dados para a unidade central de processamento. Ligados nestas interfaces pode-se ter: transmissores, chaves de pressão ou qualquer outra variável, termopares, botoeiras, etc.

Como interface de saída pode-se citar os seguintes exemplos: saídas analógica, saída digital, saída PWM, leds de sinalização, porta serial, etc. Todos eles tem por função a transformação dos dados da CPU em sinais elétricos para os atuadores ou dados que são imediatamente entendidos pelo homem. Ligados nestas interfaces pode-se ter: posicionadores eletropneumáticos, conversores de corrente para pressão, chaves magnéticas, inversores de frequência, válvulas solenoides, etc.

Estas interfaces são conectados à unidade central de processamento por intermédio de barramentos que são interfaces de comunicação dos circuitos de entrada e saída do CLP.

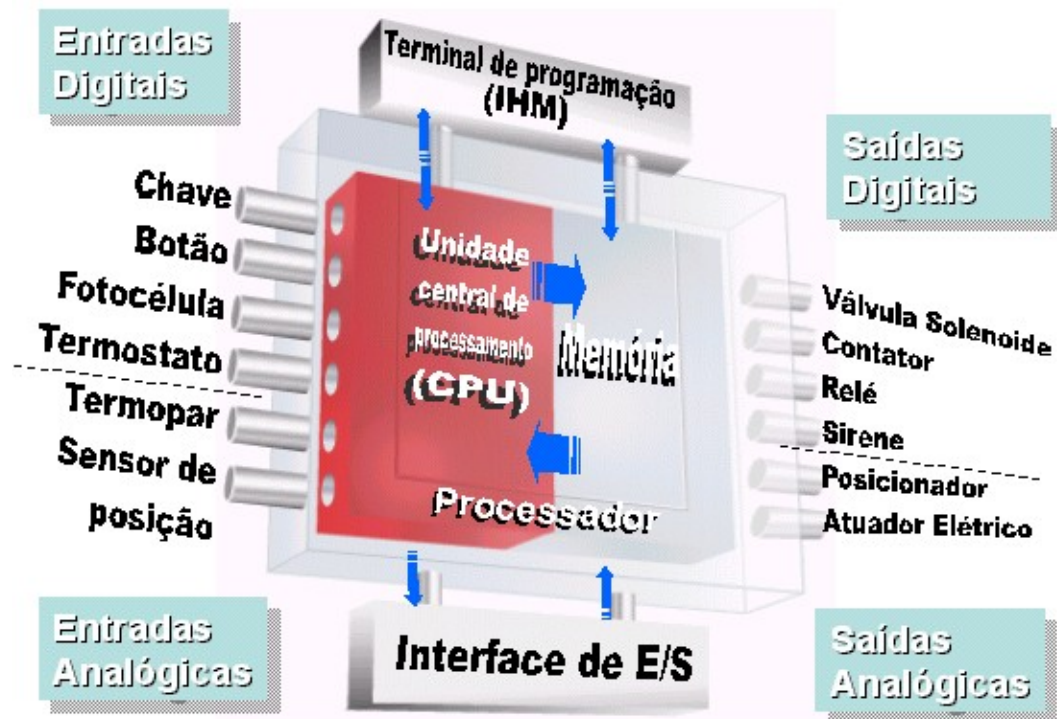


Figura 6 - Esquema da arquitetura de um CLP com interfaces de entrada e saída.

A estrutura de E/S (entradas e saídas) é encarregada de filtrar os vários sinais recebidos ou enviados para os componentes externos do sistema de controle. Estes componentes ou dispositivos no campo podem ser botões, chaves de fim de curso, contatos de relés, sensores analógicos, termopares, chaves de seleção, sensores indutivos, lâmpadas sinalizadoras, display de LEDs, bobinas de válvulas direcionais elétricas, bobinas de relés, bobinas de contadoras de motores, etc (figura anterior).

Em ambientes industriais, estes sinais de E/S podem conter ruído elétrico, que pode causar operação falha da CPU se o ruído alcançar seus circuitos. Desta forma, a estrutura de E/S protege a CPU deste tipo de ruído, assegurando informações confiáveis. A fonte de alimentação das E/S pode também constituir-se de uma única unidade ou de uma série de fontes, que podem estar localizadas no próprio compartimento de E/S ou constituir uma unidade à parte.

Os dispositivos do campo são normalmente selecionados, fornecidos e instalados pelo usuário final do sistema do CLP. Assim, o tipo de E/S é determinado, geralmente, pelo nível de tensão (e corrente, nas saídas) destes dispositivos. Os circuitos de E/S são tipicamente fornecidos pelos fabricantes de CLPs em módulos, cada um com 4, 8, 16 ou mais pontos de E/S. Além disso, a alimentação para estes dispositivos no campo deve ser fornecida externamente ao CLP, uma vez que a fonte de alimentação do CLPs é projetada para operar somente com a parte interna da estrutura de E/S e não dispositivos externos.

3.3.1 CARACTERÍSTICAS DAS ENTRADAS E SAÍDAS - E/S

A saída digital basicamente pode ser de quatro tipos: transistor, triac, contato seco e TTL podendo ser escolhido um ou mais tipos. A entrada digital pode se apresentar de várias formas, dependendo da especificação do cliente, contato seco, 24 Vcc, TTL, 110 Vca, 220 Vca, etc.

A saída e a entrada analógicas podem se apresentar em forma de corrente (4 a 20 mA, 0 a 10 mA, 0 a 50 mA), ou tensão (1 a 5 Vcc, 0 a 10 VCC, -10 a 10 VCC etc). Em alguns casos é possível alterar o range da através de software. Essas entradas um circuito especial chamado de conversor A/D que possui uma resolução em números de bits, podendo ser de 8, 10, 12, 16 ou até mais bits. A mesma premissa é válida para as saídas analógicas, mas neste caso o conversor é chamado de D/A por converter o dado digital do barramento interno em sinal analógico.

3.3.1.1 MÓDULOS OU CARTÃO DE ENTRADA

Os módulos de entrada são interfaces entre os *sensores* localizados no campo e a lógica de controle de um controlador programável. Estes módulos são constituídos de cartões eletrônicos, cada qual com capacidade para receber em certo número de variáveis conforme já citado.

Pode ser encontrado uma variedade muito grande de tipos de cartões, para atender as mais variadas aplicações nos ambientes industriais, mas apesar desta grande variedade, os elementos que informam a condição de grandeza aos cartões, são do tipo:

ELEMENTO DISCRETO: Trabalha com dois níveis definidos. Enviam o sinal discreto (binário) para o cartão ou módulo de entrada digital.

ELEMENTO ANALÓGICO: Trabalha dentro de uma faixa de valores. Enviam o sinal analógico para o cartão ou módulo de entrada analógica.

MÓDULO OU CARTÃO DE ENTRADA DIGITAL (ED)

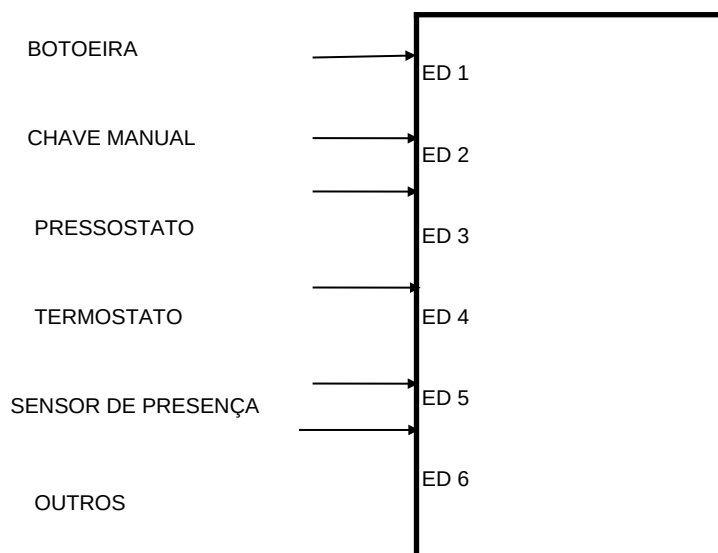


Figura 7 - Esquema do cartão ou módulo de entrada digital com respectivos elementos de campo.

A entrada digital com fonte externa é o tipo mais utilizado, também neste caso a característica da fonte de alimentação externa dependerá da especificação do módulo de entrada. Observe que as chaves que acionam as entradas situam-se no campo.

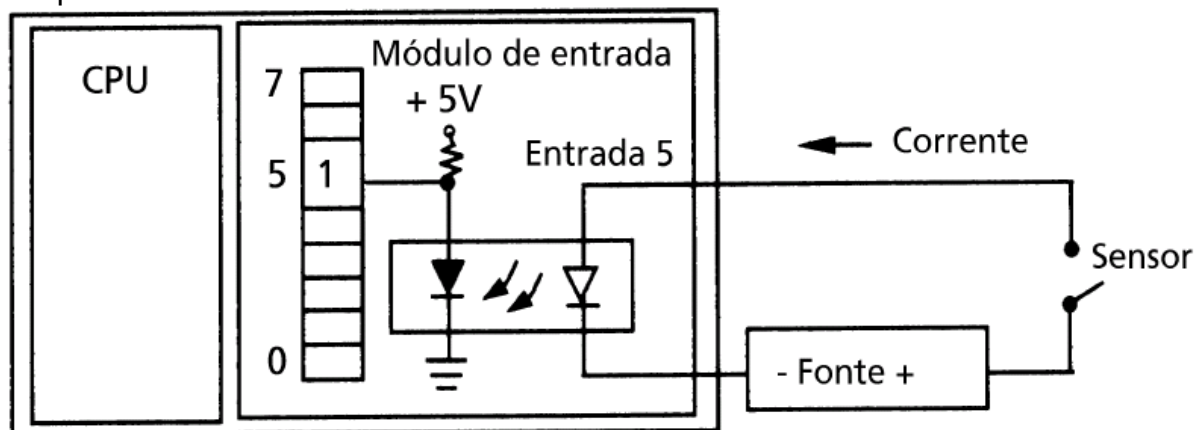


Figura 8 - Circuito de entrada digital opto isolado

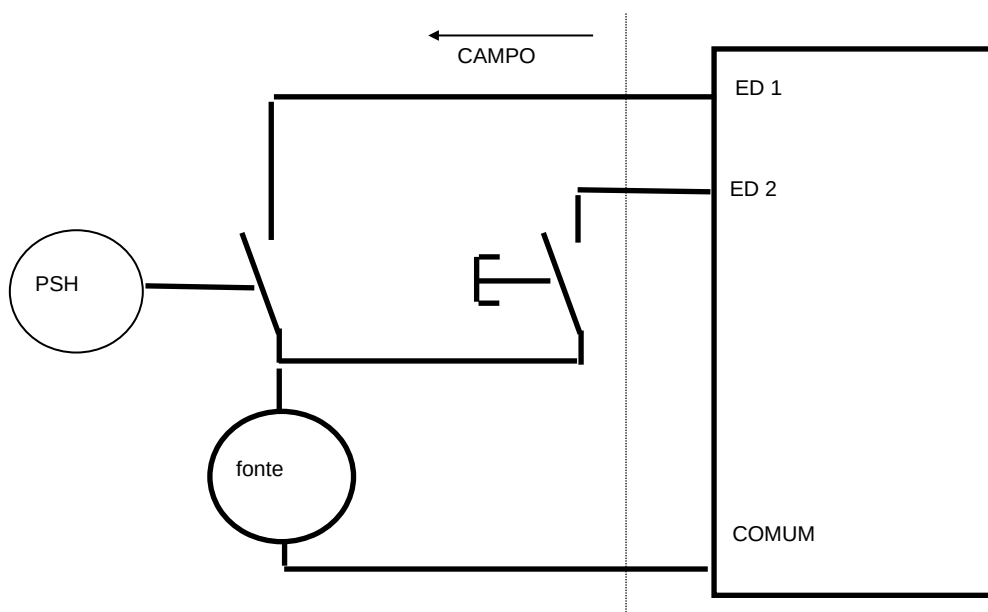


Figura 9 - Esquema do circuito elétrico de ligação de duas entradas digitais com módulo ou cartão entrada em tensão.

As entradas dos CLPs têm alta impedância e por isso não podem ser acionadas diretamente por um triac, como é o caso do acionamento por sensores a dois fios para CA, em razão disso é necessário, quando da utilização deste tipo de dispositivo de campo, o acréscimo de uma derivação para a corrente de manutenção do tiristor. Essa derivação consta de um circuito resistivo-capacitivo em paralelo com a entrada acionada pelo triac, cujos valores podem ser encontrados nos manuais do CLP, como visto abaixo.

Caso seja utilizado sensor capacitivo, indutivo ou óptico com saída à transistor com alimentação de 8 a 30 VCC, basta especificar um cartão de entrada 24 VCC comum negativo ou positivo dependendo do tipo de sensor (NPN ou PNP), e a saída do sensor será ligada diretamente na entrada digital do CLP.

A entrada digital do tipo contato seco fica limitada aos dispositivos que apresentam como saída a abertura ou fechamento de um contato. É bom lembrar que em alguns casos uma saída do sensor do tipo transistor também pode ser usada, esta informação consta no manual de ligação dos módulos de entrada

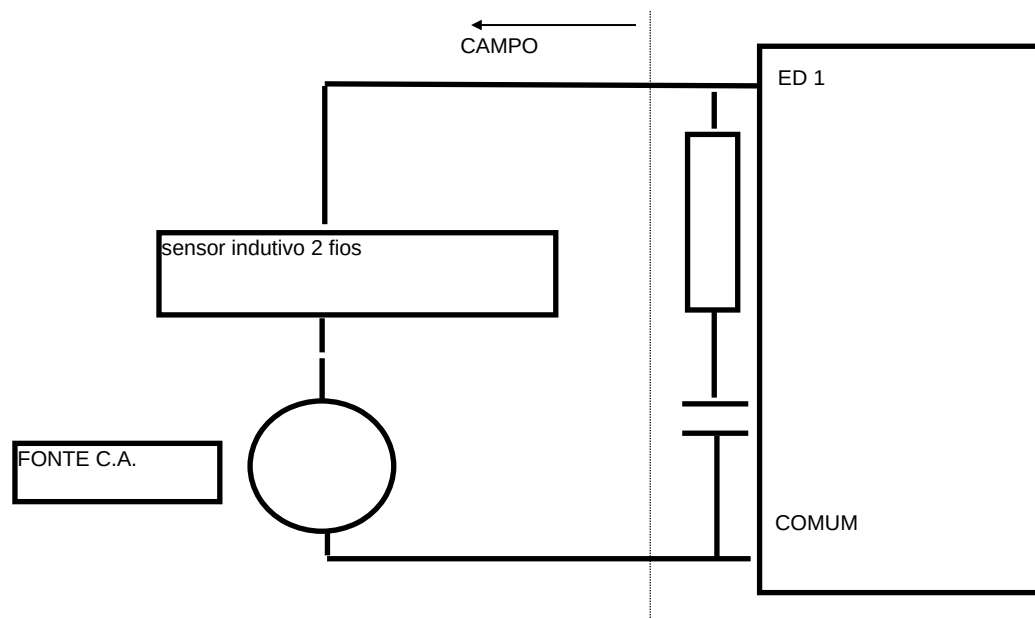


Figura 10 - Esquema do circuito elétrico de ligação de uma entradas digitais com módulo ou cartão entrada em tensão e sensor discreto indutivo.

MÓDULO OU CARTÃO DE ENTRADA ANALÓGICOS (EA)

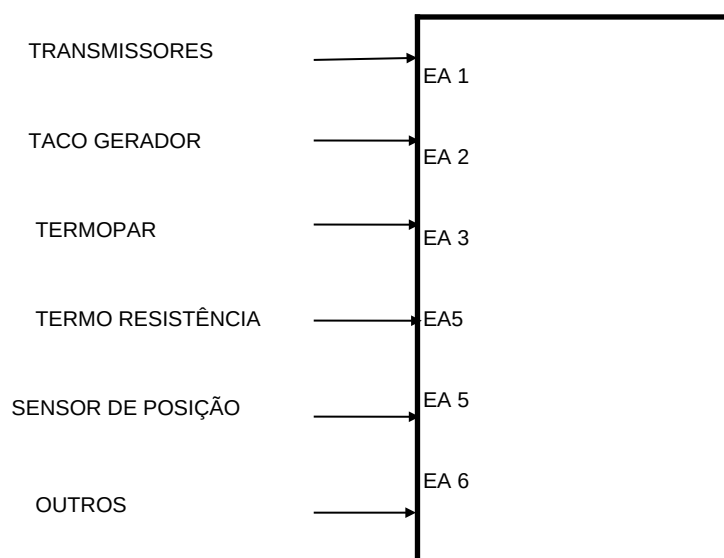


Figura 11 - Esquema do cartão ou módulo de entrada analógica com respectivos elementos de campo.

A entrada analógica em corrente é implementada diretamente no transmissor dos fios como é apresentado no diagrama.

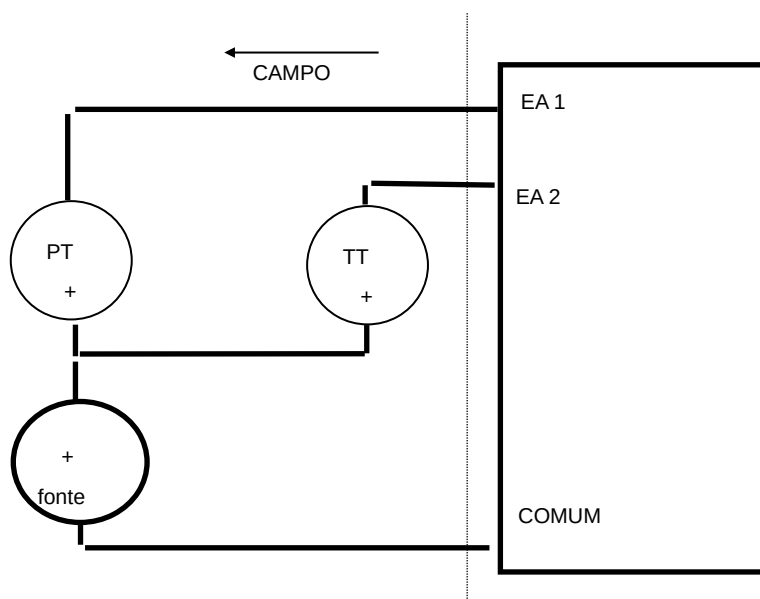


Figura 12 - Esquema do circuito elétrico de ligação de duas entradas analógicas em corrente com dois transmissores a dois fios (two wire).

A entrada analógica em tensão necessita de um shunt para a conversão do valor de corrente em tensão, como apresentado no diagrama da figura. O valor do resistor shunt dependerá da faixa de saída do transmissor e da faixa de entrada do ponto analógico, mas geralmente na indústria é utilizado resistor de 250 ohms para sinais de 4 a 20 mAcc. Porém, para o cálculo considere utiliza a lei de ohm ($R = V / I$).

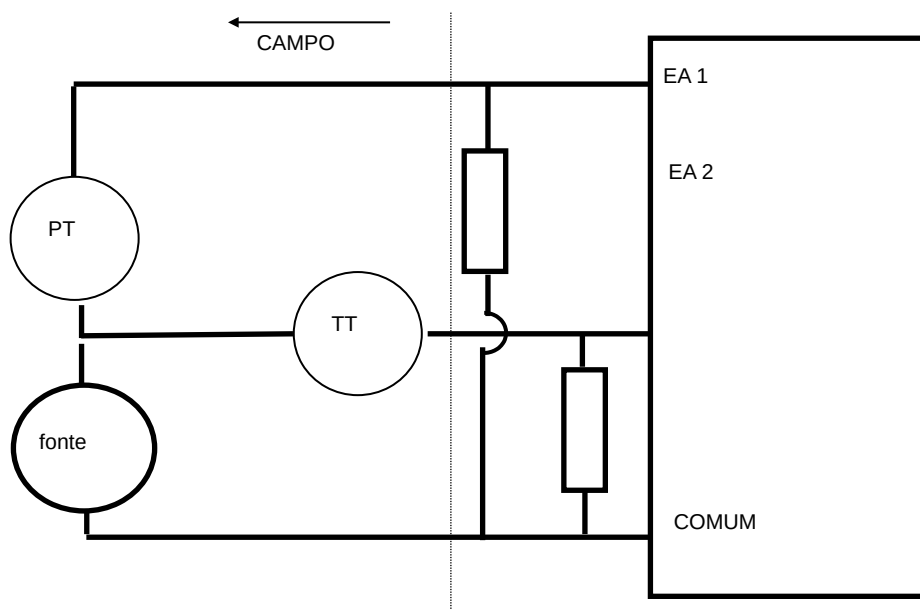


Figura 13 - Esquema do circuito elétrico de ligação de duas entradas analógicas em tensão com dois transmissores a dois fios (two wire).

TRATAMENTO DE SINAL DE ENTRADA

O tratamento que deve sofrer um sinal de entrada, varia em função de sua natureza, isto é, um cartão do tipo digital que recebe sinal alternado, se difere do tratamento de um cartão digital que recebe sinal contínuo e assim nos demais tipos de sinais.

A seguir é apresentado um diagrama onde estão colocados os principais componentes de um cartão de entrada digital de tensão alternada :

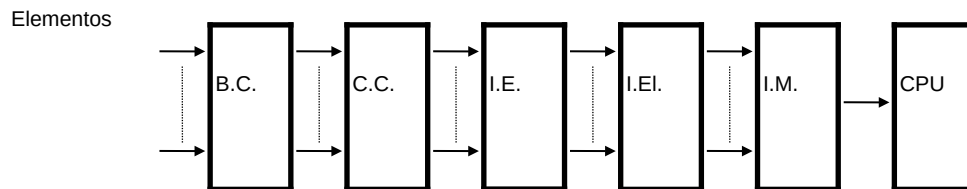


Figura 14 - Diagrama em blocos dos elementos de tratamento do sinal de entrada.

B.C. - Bornes de conexão: Permite a interligação entre o sensor e o cartão, geralmente se utiliza sistema “plug-in”.

C.C. - Conversor e Condicionador: Converte em DC o sinal AC, e rebaixa o nível de tensão até atingir valores compatíveis com o restante do circuito.

I.E. - Indicador de Estado: Proporcionar indicação visual do estado funcional das entradas.

I.El. - Isolação Elétrica: Proporcionar isolamento elétrica entre os sinais vindos e que serão entregues ao processador.

I.M. - Interface/Multiplexação: Informar ao processador o estado de cada variável de entrada.

3.3.1.2 MÓDULOS OU CARTÃO DE SAÍDA

Os módulos de saída são elementos que fazem a interface entre o processador e os elementos *atuadores*. Estes módulos são constituídos de cartões eletrônicos, com capacidade de enviar sinal para os atuadores, resultante do processamento da lógica de controle.

Basicamente, os cartões de saída irão atuar dois tipos:

ATUADORES DISCRETOS: Pode assumir dois estados definidos.

ATUADORES ANALÓGICOS: Trabalha dentro de uma faixa de valores.

MÓDULO OU CARTÃO DE SAÍDA DIGITAL

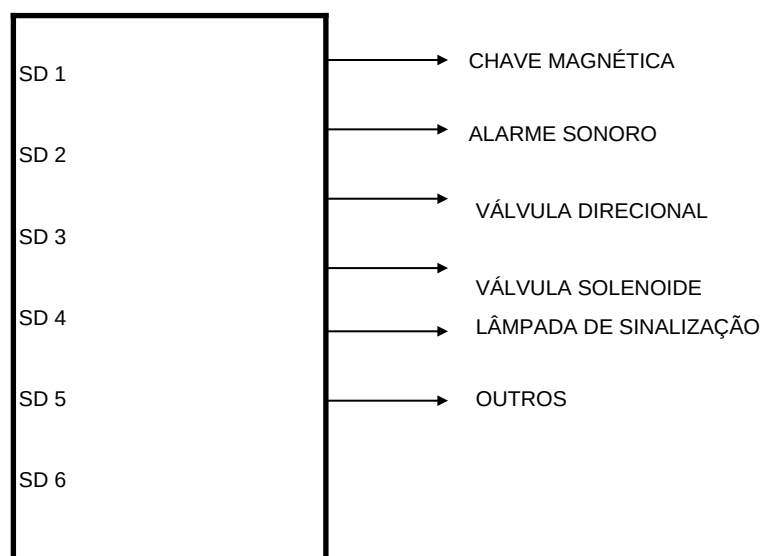


Figura 15 - Diagrama unifilar do cartão ou módulo de saída digital com exemplos de respectivos elementos de campo.

Conforme o sinal de atuação do dispositivo de campo e suas características, as saídas digitais podem ser dos seguintes tipos:

- saída digital a TRANSÍSTOR promove comutações mais velozes, mas só comporta atuadores (cargas) de tensão contínua, possuem limites de tensão e corrente inferiores comparado com a saída a contato seco;
- saída a TRIAC tem maior vida útil que o tipo a contato seco, mas só pode acionar cargas de tensão alternada;
- saída a CONTATO SECO ou RELE pode acionar cargas alimentadas por tensão contínua ou alternada, porém como qualquer dispositivo eletromecânico possui a vida útil inferior aos componentes em estado sólido. As três próximas figuras ilustram a eletrônica interna dos pontos de saída digital de diferentes tipos.

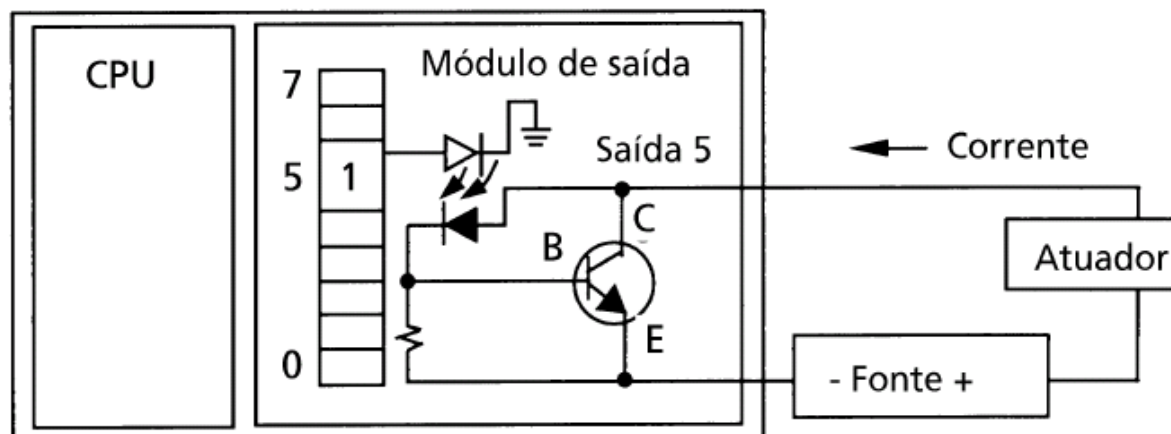


Figura 16 – Esquema do circuito interno e externo de ligação de um ponto de saída digital a transistor.

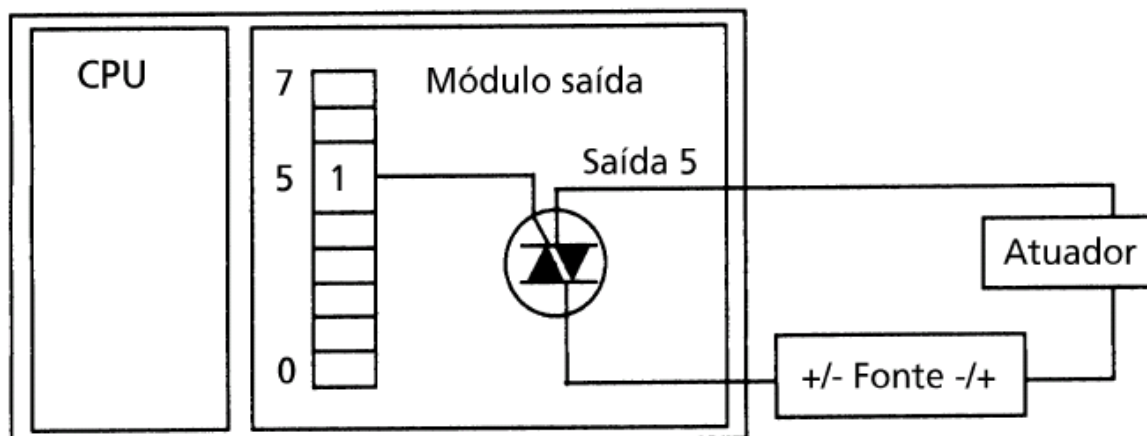


Figura 17 - Esquema do circuito interno e externo de ligação de um ponto de saída digital a triac.

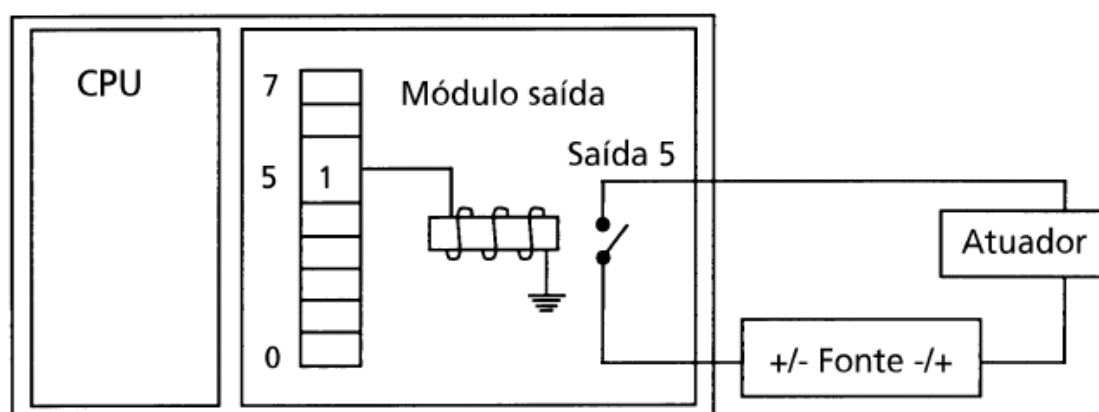


Figura 18 - Esquema do circuito interno e externo de ligação de um ponto de saída digital a rele ou contato seco.

As saídas digitais independentes possuem a vantagem de poder acionar no mesmo módulo cargas de diferentes fontes sem o risco de interligá-las. Apresentam a desvantagem de consumir mais cabos.

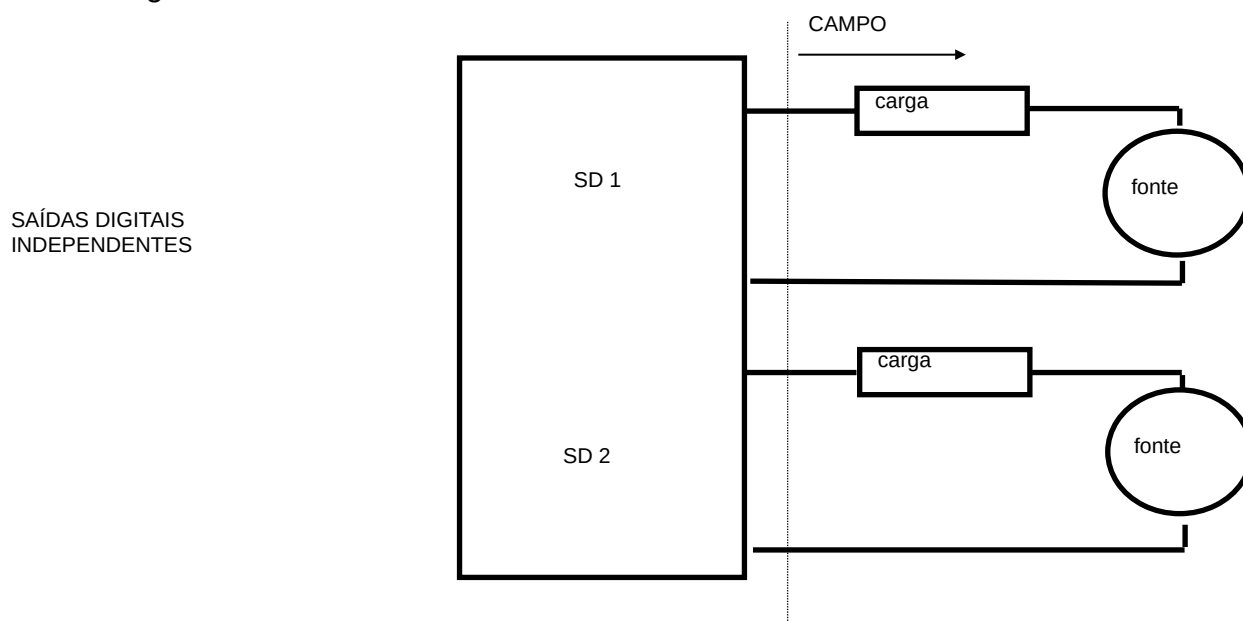


Figura 19 - Esquema do circuito elétrico de ligação de duas saídas digitais independentes.

As saídas digitais com ponto comum possuem a vantagem da simplificação dos circuitos internos do painel. Porém, esse tipo de saída é utilizada para acionamento com uso de fonte comum, ou seja, caso seja necessário acionar cargas com fontes incompatíveis entre si, será necessária a utilização de relés cujas bobinas se energizem com as saídas do CLP e cujos contatos comandem tais cargas.

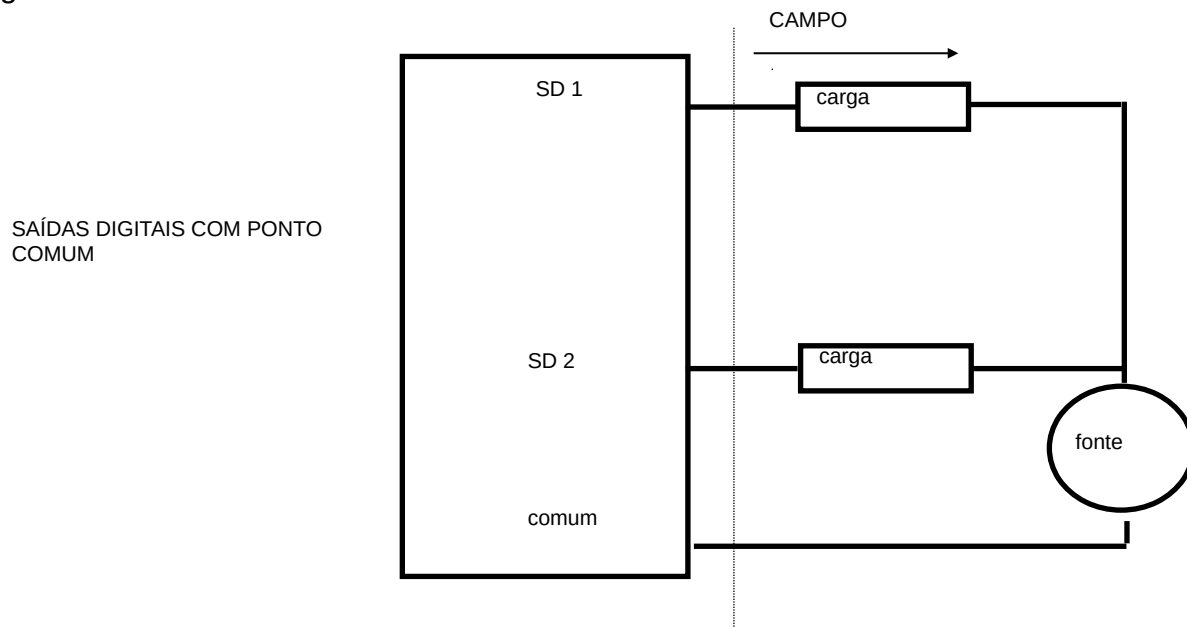


Figura 20 - Esquema do circuito elétrico de ligação de duas saídas digitais com um terminal em comum.

MÓDULO OU CARTÃO DE SAÍDA ANALÓGICA

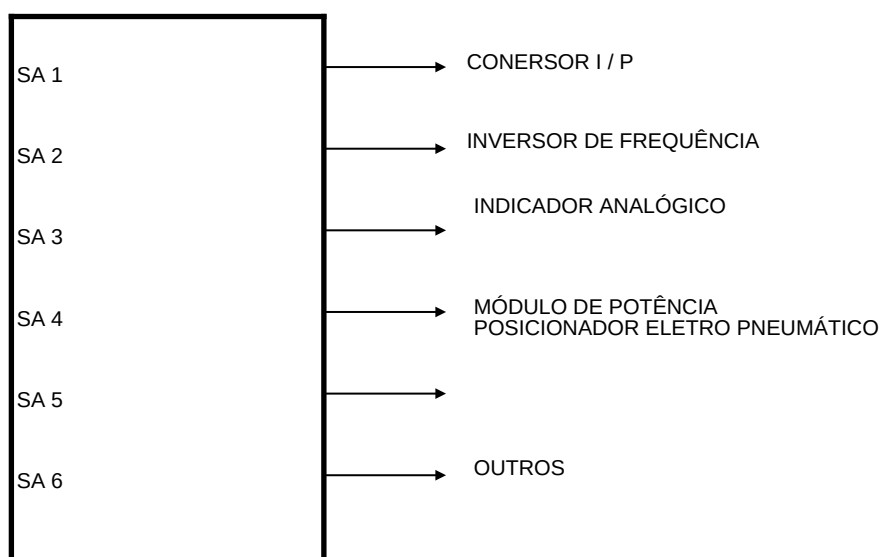


Figura 21 - Diagrama unifilar do cartão ou módulo de saída analógica com respectivos exemplos de elementos de campo.

A saída analógica em corrente ou tensão é interligada diretamente no dispositivo em questão. É bom lembrar a questão da compatibilidade dos sinais, saída em tensão só pode ser ligada no dispositivo que recebe tensão e saída em corrente pode ser ligada em dispositivo que recebe corrente ou tensão, dependendo da utilização ou não do shunt de saída.

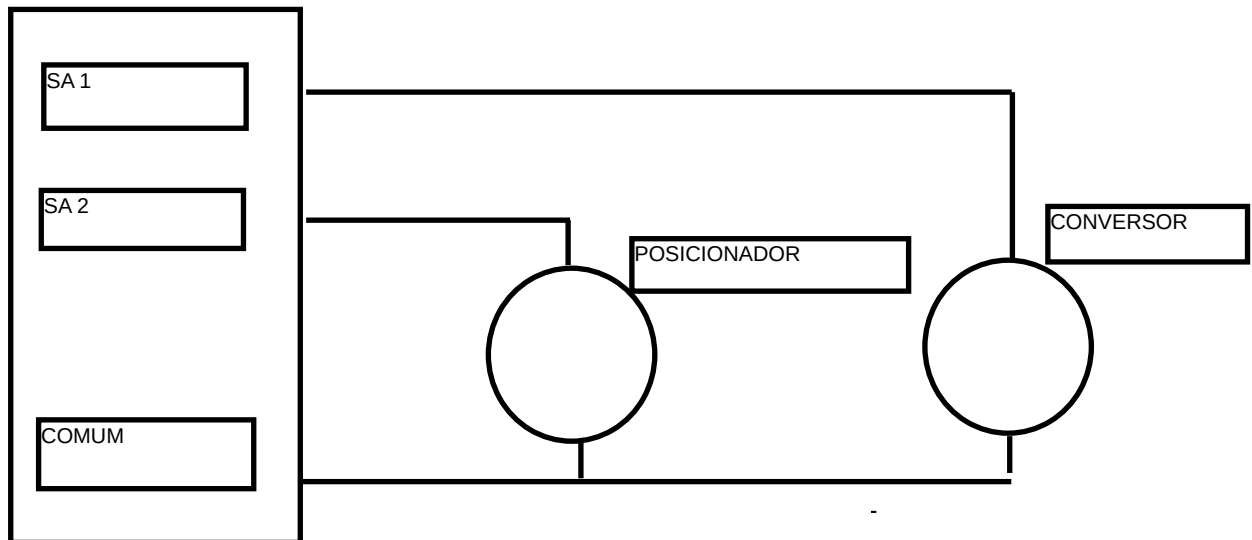
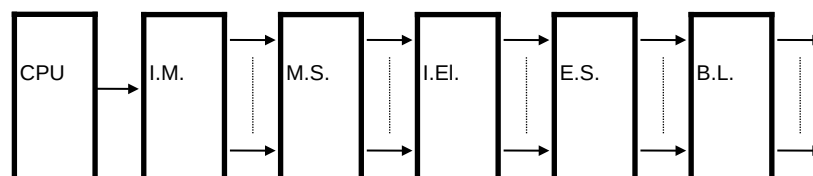


Figura 22 - Esquema do circuito elétrico de ligação de duas saídas analógicas em corrente.

TRATAMENTO DE SINAL DE SAÍDA

Existem vários tipos de cartões de saída que se adaptam à grande variedade de atuadores existentes. Por este motivo, o sinal de saída gerado de acordo com a lógica de controle, deve ser condicionado para atender o tipo da grandeza que acionará o atuador.

A seguir é apresentado um diagrama onde estão colocados os principais componentes de um cartão de saída digital de corrente contínua:



Onde:

I.M. - Interface/Multiplexação: Interpreta os sinais vindos da CPU através do barramento de dados, para os pontos de saída, correspondente a cada cartão.

M.S. - Memorizador de Sinal: Armazena os sinais que já foram multiplexados pelo bloco anterior.

I.E. - Isolação Elétrica: Proporciona isolamento elétrico entre os sinais vindos do processador e os dispositivos de campo.

E.S. - Estágio de Saída: Transforma os sinais lógicos de baixa potência, em sinais capazes de operar os diversos tipos de dispositivos de campo.

B.L. - Bornes de Ligação: Permite a ligação física entre o cartão e o elemento atuador, e utiliza também o sistema “plug-in”.

A ligação dos circuitos de entrada e ou saída é relativamente simples, dependendo apenas do tipo em questão.

Uma boa prática de todo o profissional é ler o manual de instalação dos equipamentos. No que diz respeito às saídas digitais dos CLPs devem ser rigorosamente respeitados os limites de tensão, corrente e polaridade quando for o caso.

3.3.2 TERMINAL DE PROGRAMAÇÃO

O *terminal de programação* é um dispositivo (periférico) que conectado temporariamente ao CLP, permite introduzir o programa do usuário e configuração do sistema. Pode ser um equipamento dedicado, ou seja, um terminal que só tem utilidade como programador de um determinado fabricante de CLP, ou um software executado em um computador pessoal que permite programar o CLP.

Neste periférico, através de uma ou mais linguagens será possível implementar a lógica desejada para o funcionamento do processo. Dependendo do tipo de Terminal de Programação (TP), poderão ser realizadas funções como:

- Elaboração do programa do usuário;
- Análise do conteúdo dos endereços de memória;
- Introdução de novas instruções;
- Modificação de instruções já existentes;
- Monitoração do programa do usuário;
- Verificação do estado de funcionamento do hardware do CLP;
- Atuação de saídas independente da lógica (force);
- Cópia do programa do usuário.

Os terminais de programação podem ser classificados em:

- Terminal Dedicado Portátil (legado);
- Terminal Dedicado TRC (legado);
- Terminal não Dedicado (PC executando software compatível).

TERMINAL PORTÁTIL DEDICADO (legado)

Os terminais de programação portáteis, geralmente são compostos por teclas que são utilizadas para introduzir o programa do usuário. Também conhecidos como **handheld**. Os dados e instruções são apresentados num display que fornece sua indicação, assim como a posição da memória endereçada.

A maioria dos programadores portáteis são conectados diretamente ao CLP através de uma interface de comunicação (serial). Pode-se utilizar da fonte interna do CP ou possuir alimentação própria através de bateria.



Figura 23 - Exemplo de terminal de programação dedicado portátil ou handheld utilizados em CLPs antigos (legado).

Com o advento dos computadores pessoais, estes terminais perderam sua função.

TERMINAL DEDICADO TRC (legado).

No caso do *Terminal de programação dedicado TRC* tem-se como grandes desvantagens seu custo elevado e sua baixa taxa de utilização, já que sua maior utilização se dá na fase de projeto e implantação da lógica de controle.

Estes terminais são compostos por um teclado, para introdução de dados/instruções e um monitor (TRC - tubos de raios catódicos) que tem a função de apresentar as informações e condições do processo a ser controlado.



Figura 24 - Exemplo de terminal de programação dedicado TRC (esquerda) utilizado em CLPs antigos (direita).

Como no caso dos terminais dedicados TRC, com o advento da utilização de computadores pessoais, este tipo de terminal caiu em desuso.

TERMINAL NÃO DEDICADO - PC

A utilização de um computador pessoal (PC) como terminal de programação é possível através da utilização de um software aplicativo dedicado a esta função.

Neste tipo de terminal, tem-se a vantagem da utilização de um microcomputador de uso geral realizando o papel do programador do CLP. O custo deste hardware (PC) e software são bem menores do que um terminal dedicado além da grande vantagem de ter, após o período de implantação e eventuais manutenções, o PC disponível para outras aplicações comuns a um computador pessoal.

Outra grande vantagem é a utilização de softwares cada vez mais interativos com o usuário, utilizando todo o potencial e recursos de software e hardware disponíveis neste tipo de computador.

Atualmente, a utilização dos terminais de programação portáteis (handheld) e TRC para CLP são apenas em hardware antigo e a sua manutenção é complexa.

4 PRINCÍPIO DE FUNCIONAMENTO DE UM CLP

Um *controlador lógico programável*, tem seu funcionamento baseado num sistema de microcomputador onde se tem uma estrutura de software que realiza continuamente ciclos de varredura.

4.1 ESTADOS DE OPERAÇÃO

Basicamente a CPU de um controlador programável possui dois estados de operação:

- **Programação**
- **Execução**

A CPU pode assumir também o estado de erro, que aponta falhas de operação e execução do programa.

Programação

Neste estado o CLP *não executa programa do usuário*, isto é, não assume nenhuma lógica de controle, ficando preparado para ser configurado ou receber novos programas ou até modificações de programas já instalados. Este tipo de programação é chamada **off-line**.

Execução

Estado em que o CLP assume a função de execução do programa do usuário. Neste estado, alguns controladores, podem sofrer modificações no programa. Este tipo de programação é chamada **on-line** (em linha).

4.2 FUNCIONAMENTO

Ao ser energizado, estando o CLP no *estado de execução*, o mesmo cumpre uma rotina de inicialização gravada em seu sistema operacional (firmware). Esta rotina realiza as seguintes tarefas:

- A. Limpeza da memória imagem, para operandos não retentivos;
- B. Teste de memória RAM;
- C. Teste de executabilidade do programa.

Após a execução desta rotina, a CPU passa a fazer uma varredura (ciclo) constante, isto é, uma leitura seqüencial das instruções em *loop* (laço).

Entrando no *loop*, o primeiro passo a ser executado é a leitura dos pontos de entrada. Com a leitura do último ponto, irá ocorrer, a transferência de todos os valores para a chamada memória ou tabela *imagem das entradas*.

Após a gravação dos valores na tabela imagem, o processador inicia a execução do programa do usuário de acordo com as instruções armazenadas na memória.

Terminando o processamento do programa, os valores obtidos neste processamento, serão transferidos para a chamada memória ou tabela *imagem das saídas*, como também a transferência de valores de outros operandos, como resultados aritméticos, contagens, etc.

Ao término da atualização da tabela imagem, será feita a transferência dos valores da tabela imagem das saídas, para os cartões de saída, fechando o *loop*. Neste momento é iniciado um novo *loop*. A figura a seguir ilustra o funcionamento do ciclo de operação de um CLP.

Para a verificação do funcionamento da CPU, é estipulado um tempo de processamento, cabendo a um circuito chamado de *Watch Dog Time* supervisioná-lo. Ocorrendo a ultrapassagem deste tempo máximo, o funcionamento da CPU será interrompido, sendo assumido um estado de erro.

O termo *varredura* ou *scan*, são usados para um dar nome a um ciclo completo de operação (*loop*). O tempo gasto para a execução do ciclo completo é chamado **Tempo de Varredura**, e depende do tamanho e complexidade do programa do usuário, características do processados da CPU, e a quantidade de pontos de entrada e saída.

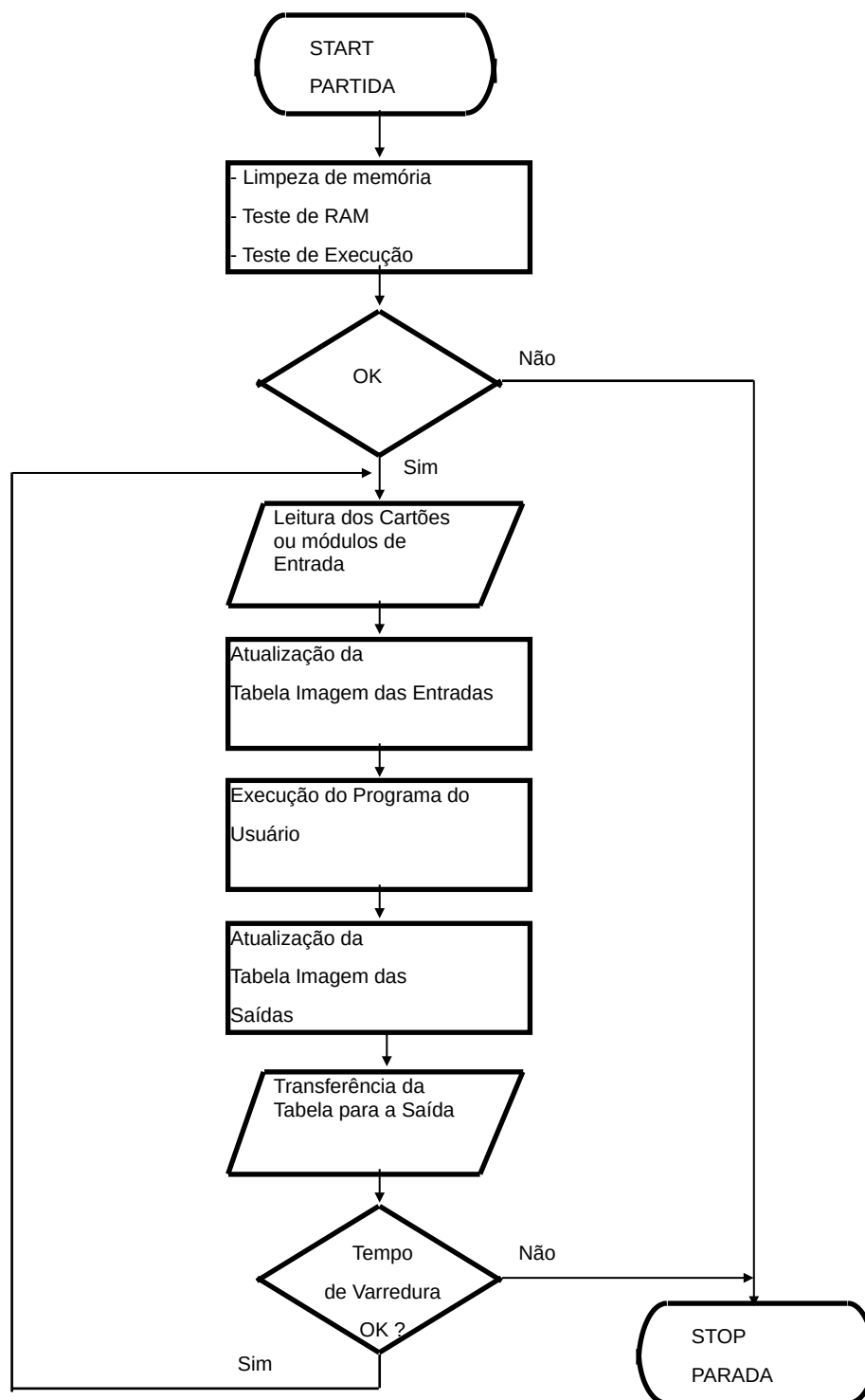


Figura 25 - Fluxograma de funcionamento do ciclo de operação de um CLP

Durante a execução do ciclo de varredura ocorre a leitura das entradas e atualização da memória imagem de E/S. A figura seguinte ilustra como esse processo funciona para entradas digitais. Estando o ponto entrada energizado (contato fechado) o bit correspondente da memória imagem ficará em nível lógico 1.

Caso o contato esteja aberto o bit correspondente na memória imagem ficará em nível lógico 0. Observe que esses estados independem se o contato de campo é normalmente aberto (NA) ou normalmente fechado (NF). Na sequência da execução do ciclo de varredura é executado o programa do usuário que, entre outros, utilizará os dados da memória imagem de E/S. Após, o resultado será escrito na tabela da memória imagem de saída.

Observando a lógica do programa do usuário apresentado na figura seguinte, pode ser observado que no estado atual das entrada (ED 00 acionada → nível lógico 1 e ED 03 desacionada → nível lógico 0) o programa acionará o bit correspondente a saída digital 04. Pode ser observado que o programa acionou a saída mesmo com uma das entradas físicas desacionadas. Isso se deve ao fato que o contato NF no programa corresponde a lógica de negação, então considerando a lógica combinacional tem-se:

$$SD03 = ED00 \cdot \overline{ED03}$$

Portanto, a saída digital 03 só será acionada quando $ED00 = 1$ E $ED03 = 0$.

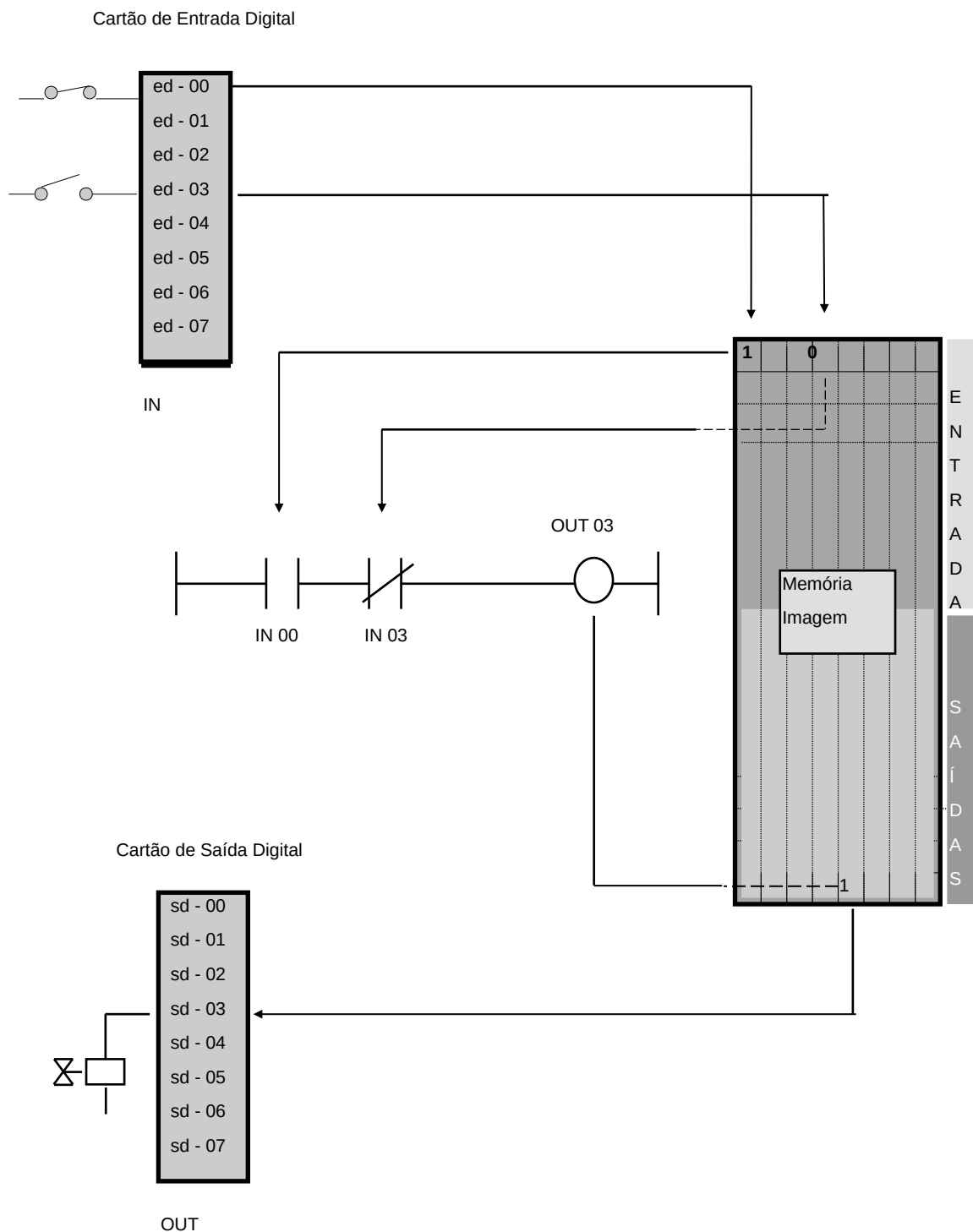


Figura 26 - Ilustração do funcionamento da atualização da memória imagem de E/S com utilização da linguagem Ladder.

5 LINGUAGEM DE PROGRAMAÇÃO

Na execução de tarefas ou resolução de problemas com dispositivos microprocessados, é necessária a utilização de uma linguagem de programação, através da qual o usuário se comunica com a máquina.

A linguagem de programação é uma ferramenta necessária para gerar o programa, que vai coordenar e sequenciar as operações que o microprocessador ou microcontrolador deve executar.

Existem linguagens consideradas Code, No Code e Low Code. As duas últimas são usadas para maximizar a produtividade da equipe dispensando um conhecimento avançado de programação, reduzindo assim a carga de trabalho que recai sobre o time de desenvolvedores.

5.1 CLASSIFICAÇÃO DAS LINGUAGENS CODE

As linguagens de programação podem ser classificadas em dois grandes grupos:

- Linguagem de baixo nível
- Linguagem de alto nível

5.1.1 LINGUAGEM DE BAIXO NÍVEL

Linguagem de Máquina

É a linguagem corrente de um microprocessador ou microcontrolador, onde as instruções são escritas em *código binário* (*bits 0 e 1*). Para minimizar as dificuldades de programação usando este código, pode-se utilizar também o *código hexadecimal*.

Código Binário

Endereço	Conteúdo
0000000000000000	00111110
0000000000000001	10000000
0000000000000010	11010011
0000000000000011	00011111
0000000000000100	00100001
0000000000000101	00000000
0000000000000111	01111110
0000000000001000	00100011
0000000000001001	10000110
0000000000001010	00111111
0000000000001011	00000001
0000000000001111	11011010
0000000000010000	00000000
0000000000010001	11011010

Código Hexadecimal

Endereço	Conteúdo
0000	3E
0001	80
0002	D3
0003	1F
0004	21
0005	00
0006	10
0007	7E
0008	23
0009	86
000A	27
000B	D3
000C	17
000D	3F

Cada item do programa, chama-se linha ou passo, representa uma instrução ou dado a ser operacionalizado.

Linguagem Assembly

Na linguagem assembly o programa é escrito com instruções abreviadas chamadas mnemônicos.

Endereço	Conteúdo
0000	MVI A,80H
0002	OUT 1FH
0004	LXI ,1000H
0007	MOV A,M
0008	INX H
0009	ADD M
000A	DAA
000B	OUT 17H
000D	MVI A,1H
000F	JC 0031H
0012	XRA A
0013	OUT 0FH
0015	HLT

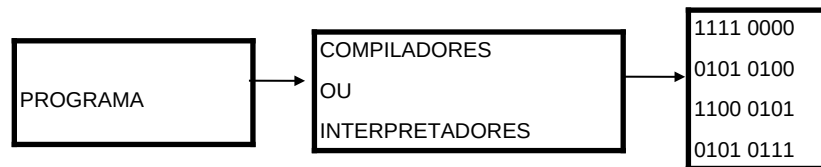
Cada microprocessador ou microcontrolador possuem estruturas internas diferentes, portanto seus conjuntos de registros e instruções também são diferentes.

5.1.2 LINGUAGEM DE ALTO NÍVEL

É uma linguagem próxima da linguagem corrente utilizada na comunicação de pessoas. No caso dos CLPs podem ser utilizadas até cinco tipos de linguagens conforme a norma IEC 61131-3. Entretanto, nos CLPs que não atendem a norma a grande maioria dos CLPs utiliza a linguagem Ladder.

Compiladores e Interpretadores

Quando um microcomputador utiliza uma linguagem de alto nível, é necessário a utilização de compiladores e interpretadores para traduzirem este programa para a linguagem de máquina.



Vantagem

Elaboração de programa em tempo menor, não necessitando conhecimento da arquitetura do microprocessador.

Desvantagem

Tempo de processamento maior do que em sistemas desenvolvidos em linguagens de baixo nível.

Exemplos de linguagens de alto nível

- Java Script;
- Python;
- C;
- C++;
- Java;
- etc.

6 PROGRAMAÇÃO DE CONTROLADORES PROGRAMÁVEIS

Conforme a IEC 61131-3, normalmente pode-se programar um controlador através de um software que possibilita desenvolver o programa em uma ou mais linguagens. Entre essas linguagens industriais estão:

- *Sequential function chart (SFC);*
- *Function blocks diagram (FBD);*
- *Ladder diagram (LD);*
- *Structured Text (ST);*
- *Instruction List (IL).*

As três primeiras linguagens listadas são consideradas gráficas e as demais textuais. Alguns CLPs, possibilitam o desenvolvimento do programa do usuário em uma ou mais formas.

É importante salientar que o software Codesys é um esforço para padronizar a IDE de programação dos CLPs. Esse software atende a norma IEC61131-3 e permite programar diversos CLPs a partir da inclusão no Codesys do firmware específico do CLP.

6.1 LADDER DIAGRAM (LD) - DIAGRAMA DE CONTATOS

Também conhecida como:

- Diagrama de relés;
- Diagrama escada;
- Diagrama "Ladder".

Esta forma gráfica de apresentação está muito próxima a normalmente usada em diagrama elétricos. Observe a figura a seguir com exemplo de código em Ladder.

Exemplo:

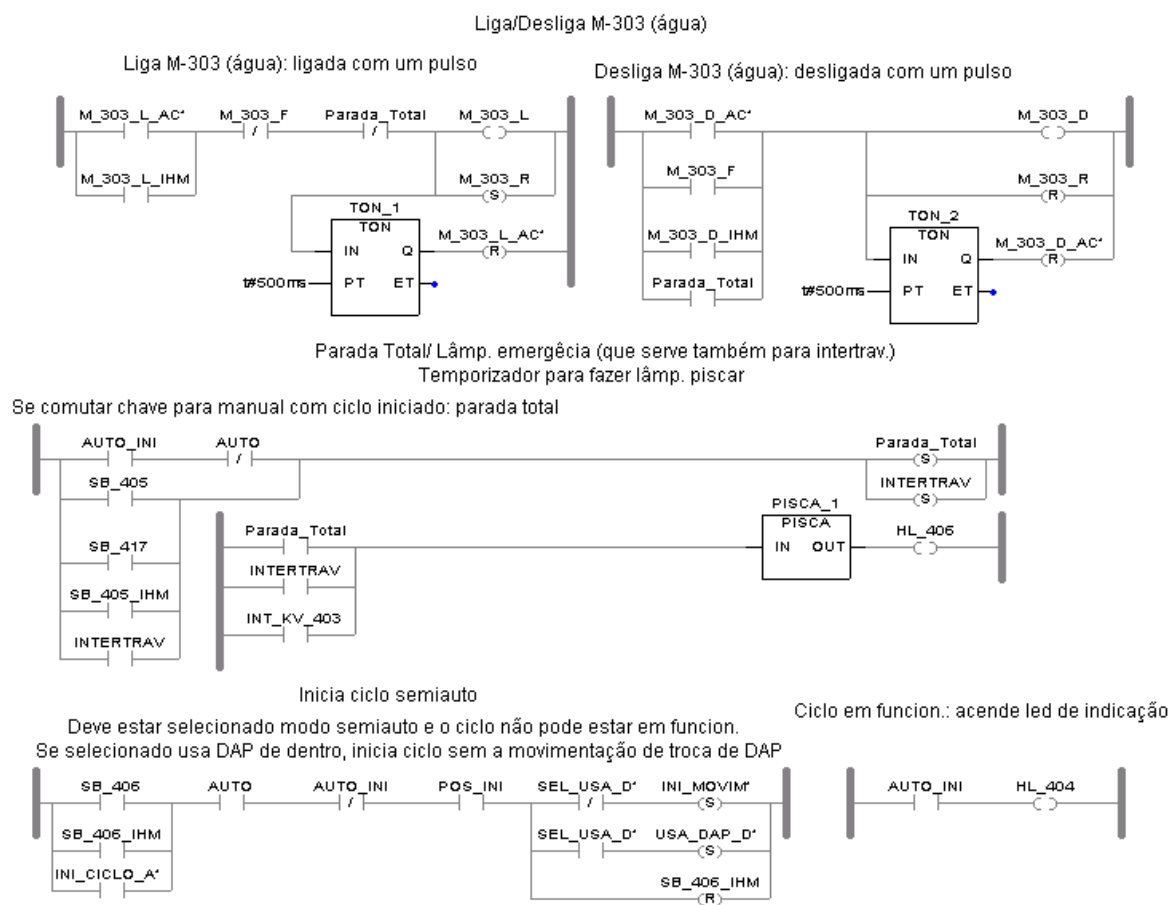


Figura 27 - Exemplo de programa em Ladder

6.2 FUNCTION BLOCKS DIAGRAM (FBD) - DIAGRAMA DE BLOCOS

Mesma linguagem utilizada em lógica digital, onde sua representação gráfica é feita através das chamadas *portas lógicas*. Observe o exemplo a seguir.

Exemplo:

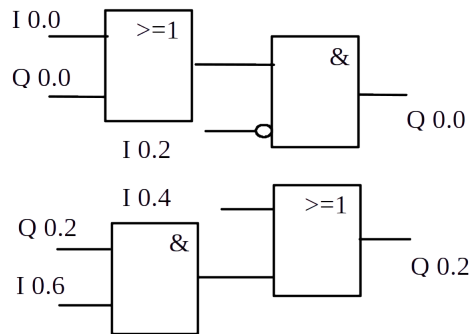


Figura 28 - Exemplo de programa em blocos

6.3 INSTRUCTION LIST (IL) - LISTA DE INSTRUÇÃO

Linguagem textual utilizada na forma de lista de comandos. Semelhante à utilizada na elaboração de programas de baixo nível.

Exemplo :

```
: A I 1.5
: A I 1.6
: O
: A I 1.4
: A I 1.3
:= Q 3.0

(I 1.5 . I 1.6) + (I 1.4 . I 1.3) = Q 3.0
```

Figura 29 - Exemplo de programa em IL

6.4 STRUCTURED TEXT (ST) – TEXTO ESTRUTURADO

Linguagem muito semelhante à utilizada na elaboração de programas para computadores em texto estruturado. Observe o exemplo a seguir.

Exemplo:

```
IF VREAL <> VREAL_ANT THEN
  VREAL_ANT := VREAL;
  V_INT_AUX := ((VREAL-VREAL_LO)/(VREAL_HI-VREAL_LO))*(V_INT_HI-V_INT_LO)
+V_INT_LO;
  V_INT := REAL_TO_INT(V_INT_AUX);
END_IF;

V_INT_AUX := INT_TO_REAL(V_INT);

IF V_INT_AUX <> V_INT_ANT THEN
  V_INT_ANT := V_INT_AUX;
  VREAL := (V_INT_AUX-V_INT_LO)*(VREAL_HI-VREAL_LO)/(V_INT_HI-V_INT_LO)+VREAL_LO;
END_IF;
```

Figura 30 - Exemplo de programa em ST

6.5 SEQUENTIAL FUNCTION CHART (SFC) - PASSOS OU STEP

Essa linguagem de programação executa rotinas baseadas em passos que são executados mediante a certas condições lógicas satisfeitas.

Exemplo:

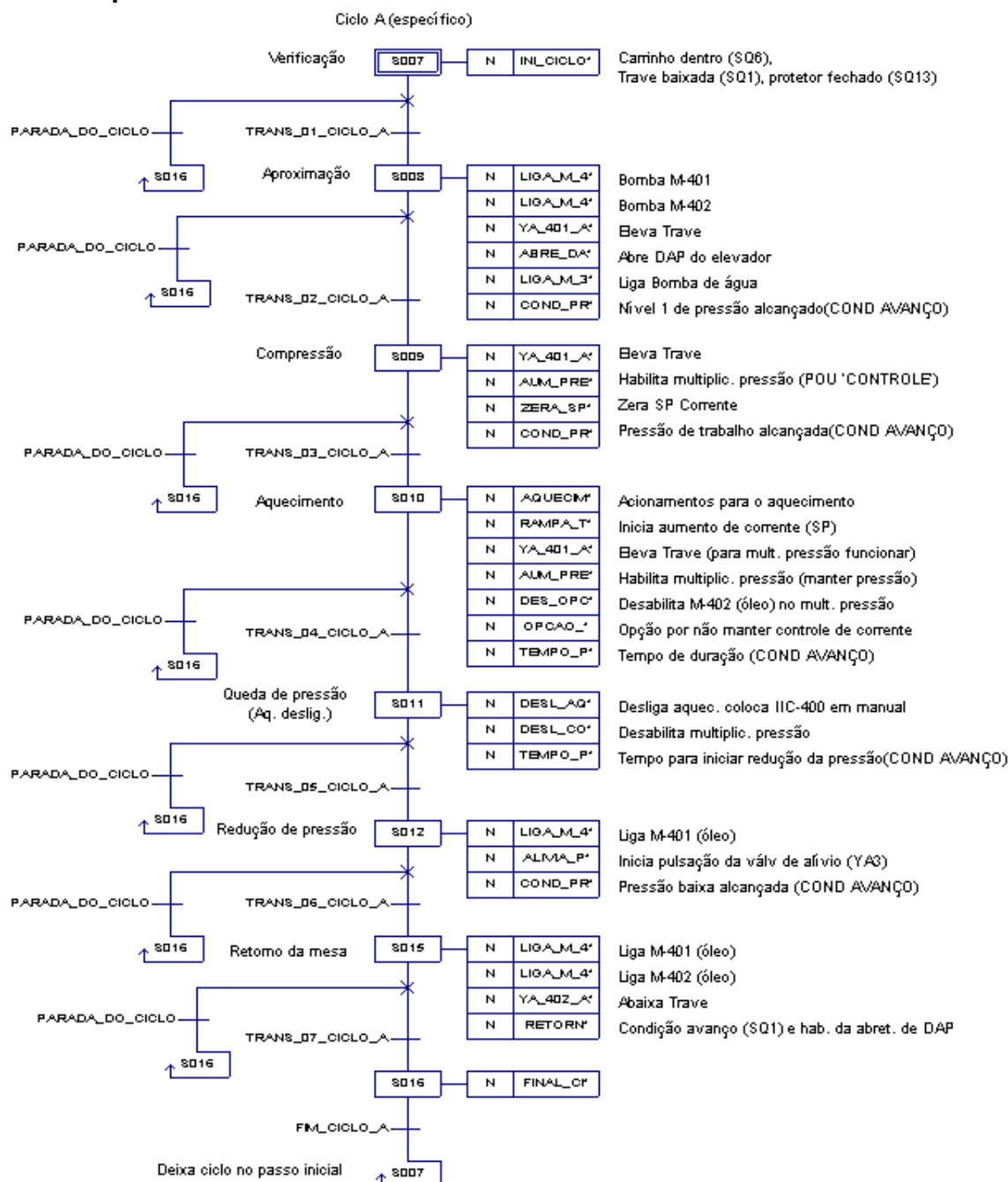


Figura 31 - Exemplo de programa em passos

6.6 ANÁLISE DAS LINGUAGENS DE PROGRAMAÇÃO

Com o objetivo de ajudar na escolha de um sistema que melhor se adapte as necessidades de cada usuário, pode-se analisar as características das linguagens programação disponíveis de CLPs.

Esta análise se deterá nos seguintes pontos:

- Quanto a forma de programação;
- Quanto a forma de representação;
- Documentação;
- Conjunto de Instruções.

Quanto a Forma de Programação

Programação Linear - programa escrito em único bloco

Programação Estruturada - Estrutura de programação que permite:

- Organização;
- Desenvolvimento de bibliotecas de rotinas utilitárias para utilização em vários programas;
- Facilidade de manutenção;
- Simplicidade de documentação e entendimento por outras pessoas além do autor do software.

Permite dividir o programa segundo critérios funcionais, operacionais ou geográficos.

Quanto a Forma de Representação

- **Diagrama de Contatos;**
 - **Diagrama de Blocos;**
 - **Lista de Instruções.**
- Estes já citados anteriormente.

Documentação

A documentação é um recurso do editor do código (IDE) do que da linguagem de programação. De qualquer forma, uma abordagem neste sentido torna-se cada vez mais importante, tendo em vista que um grande número de profissionais estão envolvidos no projeto de um sistema de automação que se utiliza de CLPs, desde sua concepção até a manutenção.

Quanto mais rico em comentários, melhor a documentação que normalmente se divide em vários níveis.

Conjunto de Instruções

É o conjunto de funções que definem o funcionamento e aplicações de um CLP. Podem servir para mera substituição de comandos a relés:

- Funções Lógicas;
- Memorização;
- Temporização;
- Contagem.

Como também manipulação de variáveis analógicas:

- Movimentação de dados;
- Funções aritméticas.

Se funções complexas de algoritmos, comunicação de dados, interfaces homem-máquina, podem ser necessárias:

- Saltos controlados;
- Indexação de instruções;
- Conversão de dados;
- PID;
- sequenciadores;
- aritmética com ponto flutuante;
- MQTT;
- Modbus;
- etc.

6.7 NORMALIZAÇÃO - IEC 61131

Para atender às demandas da comunidade industrial, um grupo de trabalho foi formado dentro da International Electrotechnical Commission (IEC) em 1979, para avaliar o projeto completo de controladores lógicos programáveis, incluindo o hardware, instalação, teste, documentação, programação e comunicação. O IEC é uma organização normativa internacional formada por representantes de todo o mundo.

Foram formados diversos grupos de trabalho (forças tarefas) para elaboração das diversas partes da norma IEC 1131. A tabela seguinte apresenta o estado da norma em 1993, quando a parte 3 chamada de IEC 1131-3 foi publicada.

Tabela 3 - Partes da norma IEC 1131

PARTE	TÍTULO	CONTEÚDO
Part 1	General Information	Definição da terminologia e conceitos.
Part 2	Equipment requirements and tests	Teste de verificação e fabricação eletrônica e mecânica.
Part 3	Programmable Languages	Estrutura do software do CLP, linguagens e execução de programas.
Part 4	User guidelines	Orientações para seleção, instalação e manutenção de CLPs.
Part 5	Messaging service specification	Funcionalidades para comunicação com outros dispositivos.
Part 6	Reservada	Reservada
Part 7	Fuzzy Control Programming	Funcionalidades de software, incluindo blocos funcionais padrões para tratamento de lógica nebulosa dentro de CLPs.
Part 8	Guidelines for the implementation of languages for programmable controllers.	Orientações para implementação das linguagens IEC 1131-3.

O IEC alterou a forma de codificação das suas normas incluindo mais um algarismo. Desta forma, a norma 1131 passou a ser denominada IEC 61131.

A norma IEC 61131 foi elaborada com o objetivo de padronizar diversos aspectos relacionados aos Controladores Programáveis (CP), assim como aplicar modernas técnicas e linguagens de programação para o desenvolvimento de software para estes sistemas. Como CP entende-se quaisquer equipamentos de controle com capacidade de programação, tais como CLP's, SDCCD's, Sistemas Híbridos, etc. A norma é composta por 8 partes, sendo que a parte 3 (61131-3) é destinada ao modelo de software e às linguagens de programação. Os principais conceitos e características apresentados pela IEC 61131-3 são:

- Base de dados com declaração de variáveis e alocação dinâmica;
- Tipagem de dados;
- Estruturação, modularização, reutilização e portabilidade de software;
- Técnicas de Orientação a Objetos;
- Processamento multitarefa;
- 5 linguagens de programação.

Apesar da norma 61131-3 introduzir modernas técnicas de programação para os sistemas de controle, esta norma é geralmente conhecida pela padronização de 5 linguagens de programação. Esta é uma interpretação inadequada por parte dos usuários de controle, pois as linguagens são simplesmente as ferramentas para implementar os conceitos do modelo de software. As 5 linguagens definidas pela norma são (figura seguinte):

- Lista de Instruções – IL;
- Diagrama Ladder – LD;
- Diagrama de Blocos Funcionais – FBD;
- Texto Estruturado –ST;
- Sequenciamento Gráfico de Funções – SFC.

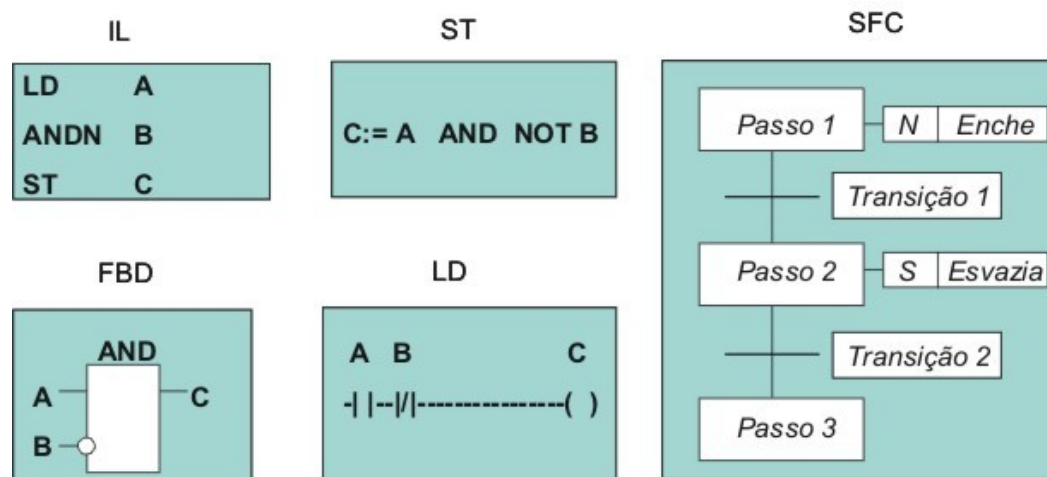


Figura 32 - Exemplos básicos de programas escritos utilizando as linguagens da norma IEC 61131-3

Uma maneira elegante de se olhar para o padrão IEC 61131-3 é dividindo-o em duas partes:

- Elementos comuns;
- Linguagens de Programação;

Analisando-se com mais detalhes cada uma destas partes:

6.7.1 ELEMENTOS COMUNS

Tipos de dados

Usado para definir o tipo de dado de qualquer parâmetro, evitando-se dividir tipos diferentes de variáveis, por exemplo. Os tipos de dados são: *booleana*, *inteiro*, *real*, *byte* e *word*, mas também *data*, *hora* e *string*. Baseado nestes tipos, é possível também definir-se um tipo de dado próprio, chamado tipo derivado de dado. Por exemplo, pode-se definir um tipo de dado como sendo *analog input channel*.

Variáveis

Variáveis podem ser atribuídas somente para explicitar endereços de hardware (entradas e saídas) em configurações, recursos e programas. Isto garante um alto nível de independência do hardware, viabilizando sua reutilização. O uso dos nomes das variáveis é normalmente limitado à POU (Unidades de Organização de Programas) em que ela foi declarada, e podem, portanto, serem reusadas em outras POU's, sem conflito. Para que uma variável seja global, deve ser declarada como tal.

6.7.2 LINGUAGENS DA NORMA IEC 61131-3

Utilizando-se das linguagens de programação, é possível implementar as Funções (Functions), Blocos Funcionais (Function Blocks) e Programas (Programs), os quais são as Unidades de Organização de Programas (POU) que incorporam o modelo de software. O devido entendimento dos conceitos envolvidos pelo modelo de software, possibilitam ao desenvolvedor aplicar as técnicas de orientação a objetos, na busca de uma estruturação de software de forma modular e reutilizável. Além disso, é possível organizar os programas em um ambiente multitarefa de forma a tirar o máximo da capacidade de processamento do controlador, de forma a atender as exigências do processo a ser controlado.

Serviços de comunicação são definidos pela parte 5 da norma e permitem a utilização de diversos blocos funcionais padronizados para a troca de dados entre controladores, interfaces de operação, softwares de supervisão e demais sistemas que necessitam interagir com os controladores. Os serviços de comunicação foram definidos dentro da filosofia Cliente Servidor e baseados na especificação MMS (Manufacturing Message Specification).

Um dos aspectos relevantes na utilização dos conceitos da norma 61131-3 consiste no encapsulamento da complexidade do algoritmo de controle. As facilidades proporcionadas pelos ambientes de programação dos modernos sistemas de controle possibilitam a implementação de algoritmos sofisticados utilizando-se as linguagens mais adequadas. Estes algoritmos são encapsulados em blocos funcionais, os quais podem ser utilizados em todas as linguagens de programação. A partir da utilização de blocos funcionais, o algoritmo encapsulado passa a ser interpretado como uma função de transferência, onde o usuário precisa se preocupar apenas com os sinais de entrada, funcionalidade do bloco e sinais de saída, não sendo necessário interpretar o algoritmo interno do bloco. Este recurso facilita em muito o entendimento da função de controle, isentando o usuário do sistema de interpretar a codificação do algoritmo. Normalmente, os ambientes de programação permitem animar os valores e estados de entrada e saída do bloco durante a execução do programa, tornando a manutenção do sistema uma tarefa

simples e rápida. Os programas são associados a tarefas com tempos de execução definidos, garantindo a execução da função de controle em tempo hábil para permitir o bom desempenho e tempo de resposta exigido pelo processo controlado.

De acordo com a norma IEC 61131-3 e considerando a forma de representação, as linguagens de programação pode ser classificadas em dois grupos: gráficas e textuais.

6.7.2.1 GRÁFICAS

Sequential Function Chart (SFC)

Descreve graficamente o comportamento sequencial de um programa de controle. É derivado de redes de Petri e do Grafcet IEC 848. O SFC estrutura a organização interna de um programa, ajudando a decompor um problema de controle em partes gerenciáveis, enquanto mantém uma visão global da solução do problema. Consiste em um conjunto de steps, ligados a blocos de ação e transições. Cada step representa um estado particular do sistema sob controle. A transição é associada com a condição, que, quando verdadeira, desativa o step anterior a ela e ativa o step seguinte. Cada elemento pode ser programado em qualquer linguagem IEC, inclusive o próprio SFC. É possível a implementação, inclusive, de sequências paralelas, como usualmente requerido em aplicações de processos batch.

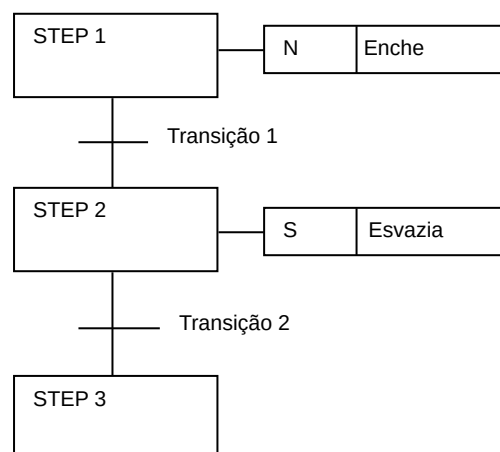


Figura 33 - Exemplo de programa básico SFC

Diagrama Ladder (Ladder Diagram – LD)

Blocos de Função (Function Block Diagram – FBD)

6.7.2.2 TEXTUAIS

Lista de Instruções (Instruction List – IL)

Texto Estruturado (Structured Text – ST)

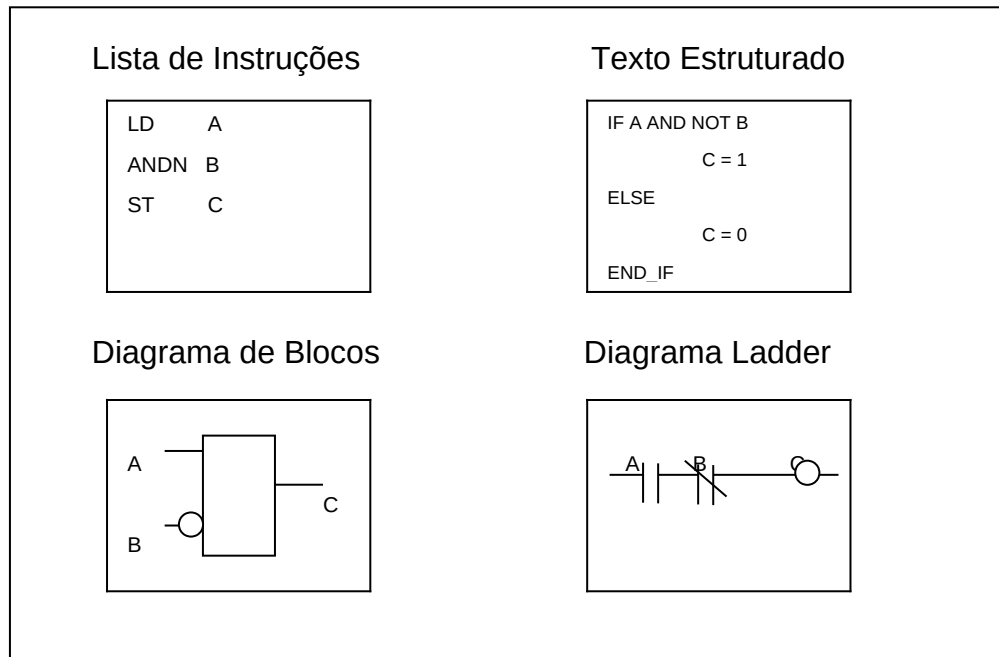


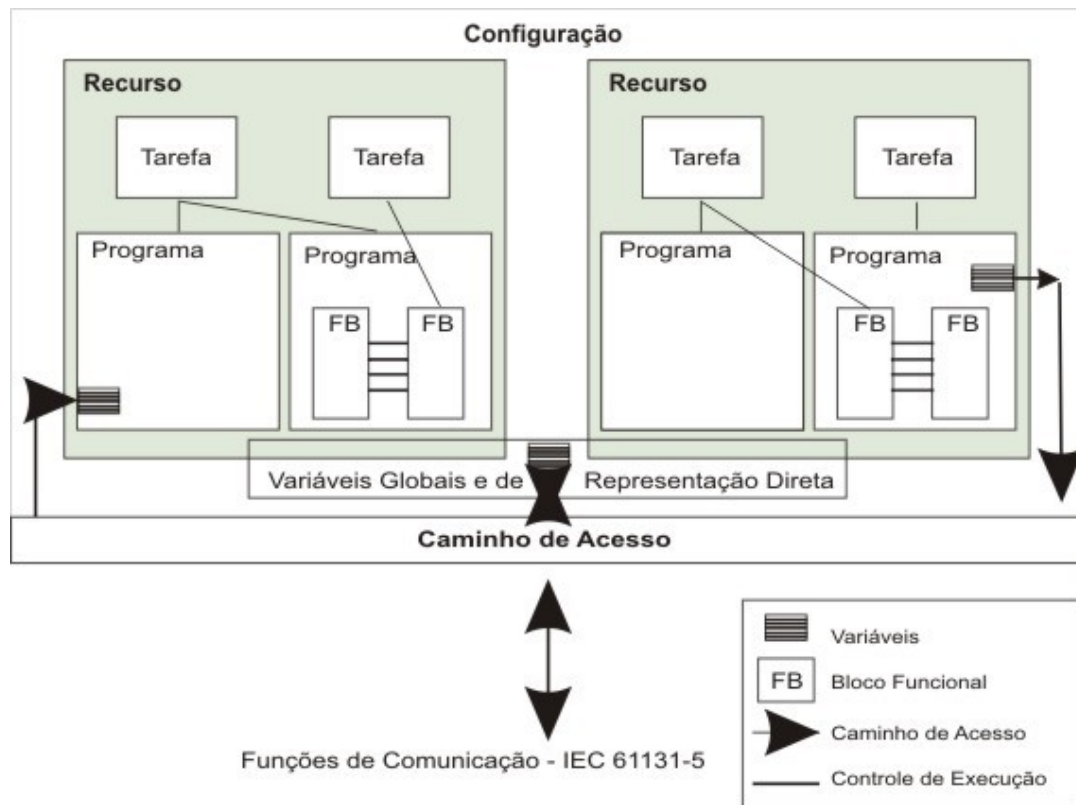
Figura 34 - Exemplo de um mesmo código implementado nas 4 linguagens IEC

A figura anterior mostra um mesmo trecho de código implementado nas quatro linguagens IEC. A escolha da linguagem de programação é dependente de:

- background o programador;
 - do problema a ser tratado;
 - do nível de descrição do problema;
 - da estrutura do sistema de controle;
 - da interface para outros departamentos/pessoas;
- Diagrama Ladder, conforme já apresentado, tem suas raízes nos EUA. É baseado na representação gráfica de lógica de relés em escada. Ladder é a linguagem mais utilizada para a implementação de códigos em CLPs.
 - Lista de instruções é a contraparte europeia. Como linguagem textual é semelhante ao assembler.
 - Diagrama de blocos é muito comum para a indústria de processos. Ele expressa o comportamento de funções, blocos de funções e programas como um conjunto de blocos gráficos interconectados., como em um desenho de circuito eletrônico. Assemelha-se à representação de um sistema em termos do fluxo de sinais entre os elementos de processamento.
 - Texto estruturado é uma linguagem muito poderosa com suas raízes em Ada, Pascal e 'C'. Pode ser usada na definição de blocos de função complexos, que podem ser utilizados com quaisquer outras linguagens, e no detalhamento das ações e transições de um programa SFC.

6.7.2.3 CONFIGURAÇÃO, RECURSOS E TAREFAS

Para entender melhor estes conceitos, observe a figura seguinte que mostra o modelo de software proposto pelo padrão IEC 61131-3.



9290-707-06-001 (Rev.:00)

Figura 35 - Modelo de software proposto pelo padrão IEC 61131-3

Configuração (configuration): formulação de um software completo, requerido para resolver um problema particular de controle. Uma configuração é especificada para um tipo particular de sistema de controle, incluindo os recursos de hardware. Para uma dada configuração pode-se definir um ou mais recursos (resources).

Recursos (resources): corresponde a uma facilidade de processamento que é capaz de executar programas baseados no padrão IEC. Para um dado recurso uma ou várias tarefas podem ser definidas.

Tarefas (tasks): controlam a execução de um conjunto de programas e/ou blocos de função. Podem ser executadas periodicamente ou na ocorrência de algum evento (trigger), como, por ex., a mudança de uma variável para uma região limite.

Programas (programs): construídos a partir de elementos diferentes de software, escritos em qualquer linguagem definida pelo padrão IEC.

6.7.2.4 UNIDADES DE ORGANIZAÇÃO DE PROGRAMAS (PROGRAM ORGANIZATION UNITS - POU)

- **Funções (Functions)**

IEC define algumas funções padrão (ADD, ABS, SQRT, SIN, COS) e funções definidas pelo usuário.

- **Blocos de funções (Functions blocks diagrams - FBD)**

São equivalentes a circuitos integrados, CIs, representando uma função de controle especializada. Eles contêm dados e algoritmo, o que equivale a dizer que possuem memória passada (o que consiste em uma das diferenças entre uma FBD

e uma função). Como um CI ou uma caixa preta, eles possuem uma interface bem definida. Permite separar bem os níveis de programação e manutenção. Ex.: PID. São altamente reutilizáveis.

- **Programas (Programs)**

Tipicamente, um programa consiste em uma rede de funções (functions) e blocos de função (function blocks), que podem trocar dados. Funções e blocos de funções são blocos de construção, básicos, contendo uma estrutura de dados e um algoritmo.

7 PROGRAMAÇÃO EM LADDER

É extremamente importante entender que atualmente existe uma tendência da padronização das linguagens de programação para os novos CLPs. Entretanto, no mercado existem CLPs que não atendem a IEC61131, pois são anteriores a norma ou por razões comerciais os fabricantes decidiram não seguir a norma. Na maioria dos casos a IDE de programação é específica. Porém o software Codesys é um esforço que busca a padronização da IDE.

O diagrama Ladder utiliza lógica de relé, com contatos (ou chaves) e bobinas, e por isso é a linguagem de programação de CLP mais simples de ser assimilada por quem já tenha conhecimento de circuitos de comando elétrico.

Compõe-se de vários circuitos dispostos horizontalmente, com a bobina na extremidade direita, alimentados por duas barras verticais laterais. Por esse formato é que recebe o nome de **Ladder** que significa escada, em inglês.

Cada uma das linhas horizontais é uma sentença lógica onde os contatos são as entradas das sentenças, as bobinas são as saídas e a associação dos contatos é a lógica.

São os seguintes os símbolos básicos:

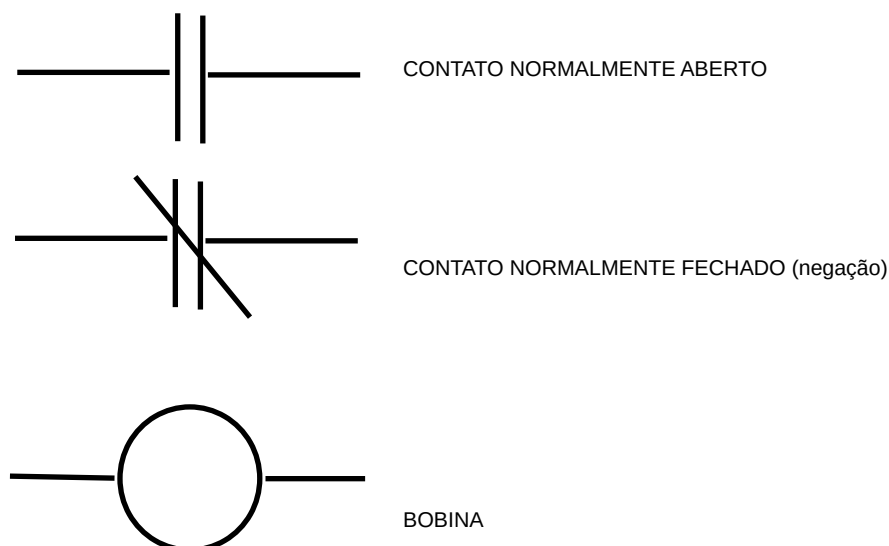


Figura 36 Elementos básicos binários da programação em Ladder

No Ladder cada operando (nome genérico dos contatos e bobinas no Ladder) é identificado com um endereço da memória à qual se associa no CLP. Esse endereço aparece no Ladder com um nome simbólico, para facilitar a programação, arbitrariamente escolhido pelo fabricante como os exemplos vistos a seguir.

Outros tipos de endereçamento; 125/04 (1 = entrada, 2 = gaveta, 5 = número do cartão ou módulo, 04 = número do ponto), 013/01 (0 = saída, 1 = número da gaveta, 3 = número do módulo, 01 = número do ponto).

Tabela 4 - Tabela de alguns CLPs X exemplo de endereçamento simbólico

FABRICANTE	MODELO	E.D.	S.D.	E.A.	S.A.	BIT AUX.	PALAVRA	PALAVRA DO SISTEMA	CONTADOR / TEMPORIZADOR
GEFANUC	90-70 90-30 90-20 90-MICRO	%I1 a %I...	%Q1 a %Q...	%AI a %AI...	%AQ1 a %AQ...	%M1 a %M... %T1 a %T...	%R1 a %R...	%S	%Rx x x+1 x+2 PARA CADA
ALLEN BRADLEY	SLC-500	I:SLOT .PONT O I:1/0 a I:...	O:SLOT T.PON TO O:1/0 a O:...	I:SLOT. PONTO I:3.0 a I:3....	O:SLOT T.PON TO O:3.0 a O:3....	B3:0/0 a B3:...	N7:0 a N7:...	S: R6:0 a R6:...	T4:0 A T4:.. C5:0 A C5:...
ALTUS	AL500	R0 a R...	R60 a R...	-	-	A0 a A...	M0 a M...	-	M0 PARA CADA
ALTUS	PICOLLO	%E0.0 a %E...	%S2.0 a %S...	%M	%M	%A0.0 a %A...	%M0 a %M...		%M0 PARA CADA
FESTO	FPC101 FPC103	I0.0 a I...	O0.0 a O...	I10 a I13 OU IU0 a IU3	OU0 e OU1	F0.0 a F15.15	R0 a R64	FW0 a FW15	T0 a T31 C0 a C15

Para efeito de convenção na explicação os endereços serão identificados como:

ED - para entrada digital;

EA - para entrada analógica;

SD - para saída digital;

SA - para saída analógica.

A - para bit auxiliar.

Os bits auxiliares de memória (A) é constituído de bits da memória de dados que podem ser manipulados pela programação. De forma geral, esses bits auxiliam na programação equivalendo no circuito elétrico aos relés auxiliares. Fundamentalmente não constituem entradas ou saídas digitais, pois não fazem parte da memória imagem de I/O.

O estado de cada operando é representado em um bit correspondente na memória imagem: este bit assume nível **1** se o operando estiver acionado e **0** quando desacionado.

As bobinas acionam o seu endereço

Enquanto uma bobina com endereço de saída estiver acionada, um par de terminais no módulo de saída será mantido em condição de condução elétrica.

Os contatos se acionam pelo endereço que os identifica.

Os contatos endereçados como entrada se acionam enquanto seu respectivo par de terminais no módulo de entrada é acionado: fecham-se se forem NA e abrem-se se forem NF.

Com relação ao que foi exposto acima sobre os contatos endereçados como entrada, os que tiverem por finalidade acionar ou energizar uma bobina deverão ser do mesmo tipo do contato externo que aciona seu respectivo ponto no módulo de entrada.

Já os que forem usados para desacionar ou desenergizar uma bobina devem ser de tipo contrário do contato externo que os aciona. Abaixo vê-se um quadro elucidativo a esse respeito.

	Se a chave externa for	o contato no Ladder deve ser
Para ligar	NA	NA
	NF	NF
Para desligar	NA	NF
	NF	NA

Percebe-se pois que pode ser usada chave externa de qualquer tipo, desde que no Ladder se utilize o contato de tipo conveniente. Mesmo assim, por questão de segurança, não se deve utilizar chave externa NF para ligar nem NA para desligar. Sempre deve-se buscar a condição de falha segura, ou seja, colocar o processo na condição mais segura no caso de falha.

Outra forma para definir se o contato no Ladder será NA ou NF é entender que o contato NF também é conhecido como negação, ou seja, campo fechado (Ladder NF) o resultado é aberto e campo aberto (Ladder NF) o resultado é fechado.

7.1 DESENVOLVIMENTO DO PROGRAMA LADDER

No desenvolvimento de um sistema independentemente do método ou da técnica, deve-se ter um conhecimento fundamentado da tecnologia considerada, das possibilidades de ligação, e das características dos elementos utilizados.

Deve ser muito bem lembrado que os equipamentos podem causar danos as máquinas ou processos por eles controlados, no caso de defeito de sua partes e peças, erros de programação ou instalação, podendo inclusive colocar em risco vidas humanas.

Deve-se analisar as possíveis consequências destes defeitos e providenciar instalações adicionais externas de segurança do sistema, principalmente nos casos de instalação inicial e testes.

Portanto todo o projeto deve ser executado de maneira a se obter os seguintes requisitos:

- Segurança de pessoas;
- Segurança da instalação;
- Garantia de funcionamento;
- Facilidade de manutenção;
- Facilidade de operação;
- Custo.

Outro aspecto a ser considerado são as condições marginais de funcionamento e segurança, como:

- Ciclo único / Ciclo contínuo;
- Partida / Parada;
- Manual / Automático;
- Parada de emergência / Desbloqueio.

Estas condições devem ser introduzidas no sistema, somente depois de esquematizado o ciclo básico.

Após a definição da operação de um processo onde são geradas as necessidades de sequenciamento e/ou intertravamento, esses dados e informações são passados sob forma de diagrama lógico, diagrama funcional ou matriz de causas e efeitos e a partir daí o programa é estruturado.

A seguir vêm-se os passos para elaboração do programa que irá controlar um processo ou equipamento.

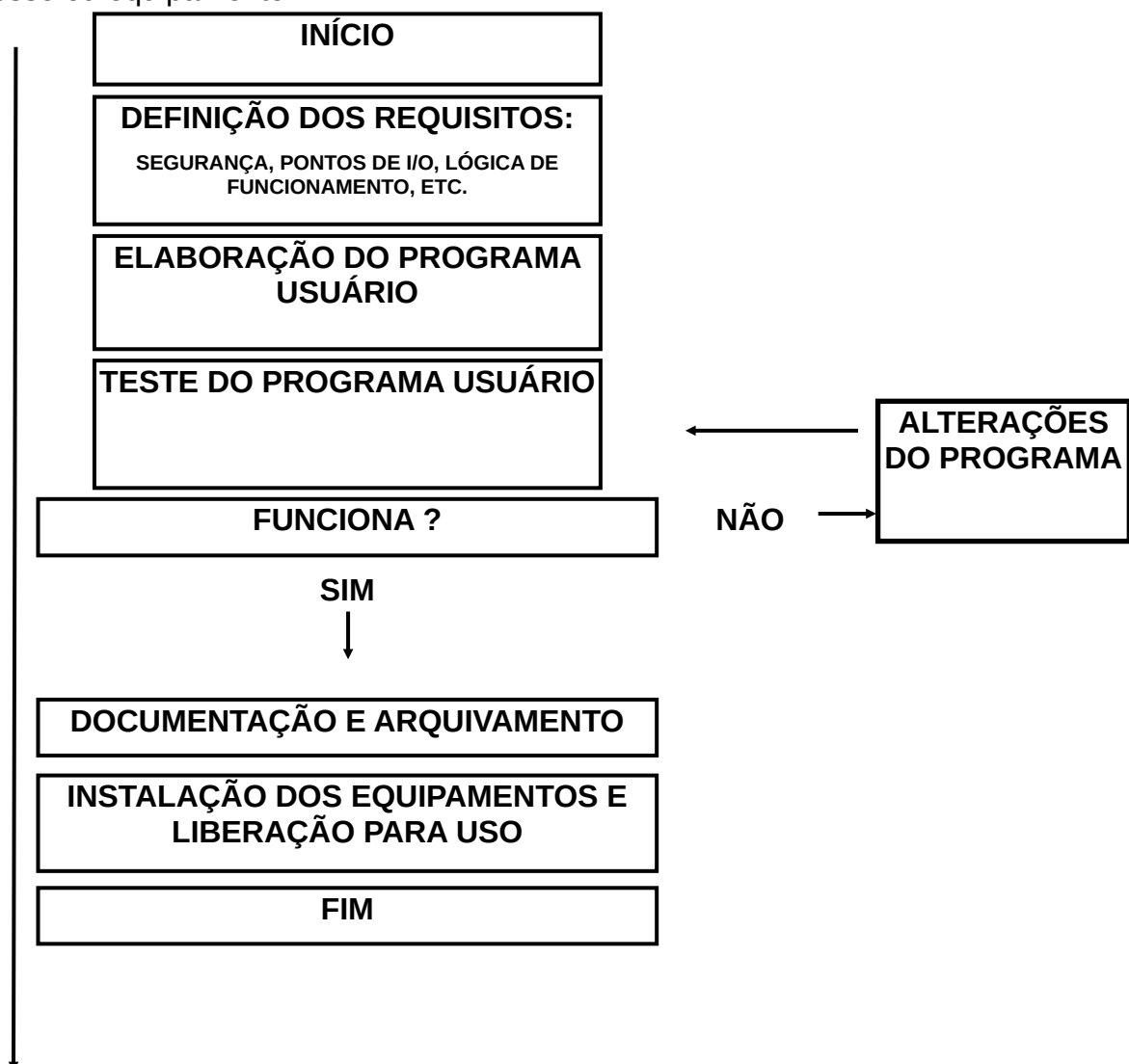


Figura 37 - Passos para elaboração do programa que irá controlar um processo ou equipamento.

A lógica de diagrama de contatos do CLP assemelha-se à de relés. Para que um relé seja energizado, necessita de uma continuidade elétrica, estabelecida por uma corrente elétrica.

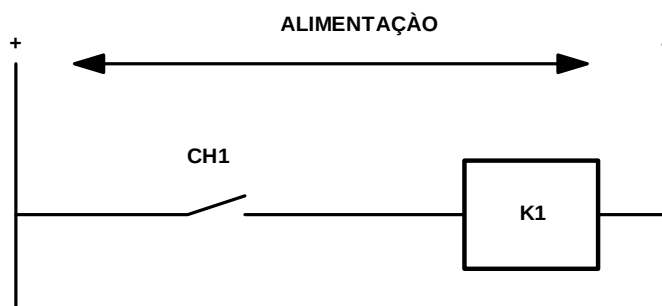


Figura 38 - Circuito básico de acionamento.

Ao ser fechada a CH1, a bobina K1 será energizada, pois será estabelecida uma continuidade entre a fonte e os terminais da bobina.

O programa equivalente do circuito anterior, na linguagem Ladder, será o seguinte.

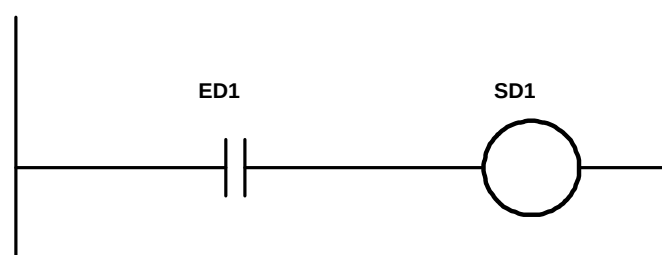


Figura 39 - Circuito básico de acionamento em Ladder

Analisando os módulos de entrada e saída do CLP, quando o dispositivo ligado à entrada digital ED1 fechar, este acionará o contato ED1, que estabelecerá uma continuidade de forma a acionar a bobina SD1, conseqüentemente o dispositivo ligado à saída digital SD1 será acionado.

Uma prática indispensável é a elaboração das tabelas de alocação ou mapa de memória dos dispositivos de entrada/saída. Esta tabela é constituída do nome do elemento de entrada/saída, sua localização e seu endereço de entrada/saída no CLP. Exemplo:

TAG	DESCRIÇÃO	LOCALIZAÇÃO	ENDEREÇO
PSL - 100	Chave de pressão baixa	Topo do tanque pressurizado 2	ED1
TT - 400	Transmissor de temperatura	Saída do misturador	EA1
FS - 200	Chave de fluxo	Saída de óleo do aquecedor	ED2
TV - 400	Válvula de temperatura	Ao lado do tanque 23	SD1

O NF é um contato de negação ou inversor, como pode ser visto no exemplo abaixo que é similar ao programa anterior substituindo o contato NA por um NF.

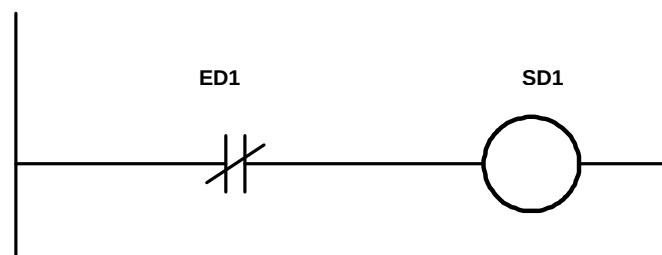


Figura 40 - Linha Ladder com contato inversor

Analisando os módulos de entrada e saída, quando o dispositivo ligado a entrada digital ED1 abrir, este desacionará o contato ED1, este por ser NF estabelecerá uma continuidade de forma a acionar a bobina SD1, conseqüentemente o dispositivo ligado à saída digital SD1 será acionado. A seguir tem-se o gráfico lógico referente aos dois programas apresentados anteriormente.

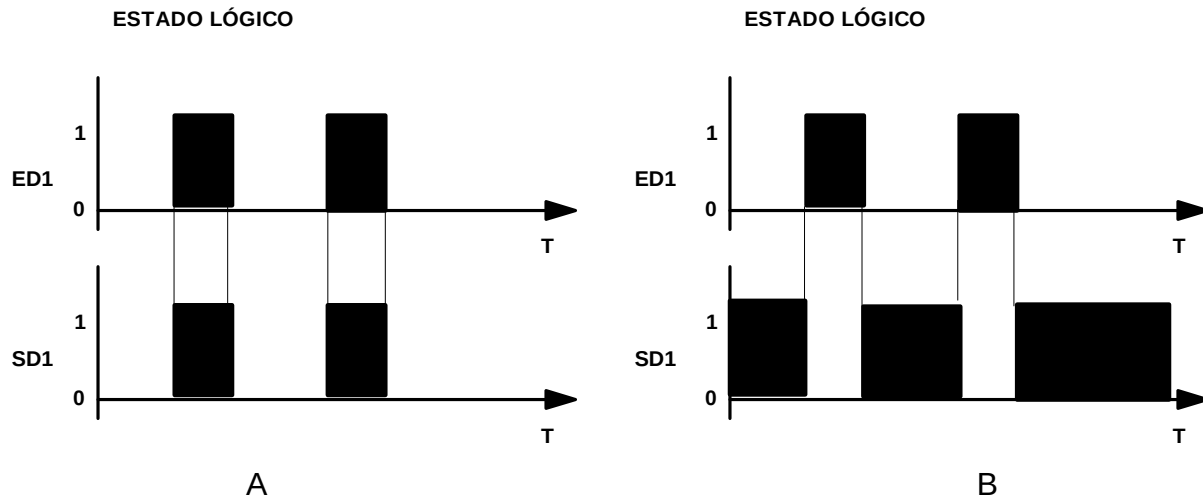


Figura 41 - Ladder utilizando contato normalmente aberto (A) e Ladder utilizando contato normalmente fechado (B)

7.1.1 ASSOCIAÇÃO DE CONTATOS NO LADDER.

No Ladder se associam contatos para criar as lógicas **E** e **OU** com a saída.

Os contatos em série executam a lógica **E**, pois a bobina só será acionada quando todos os contatos estiverem fechados

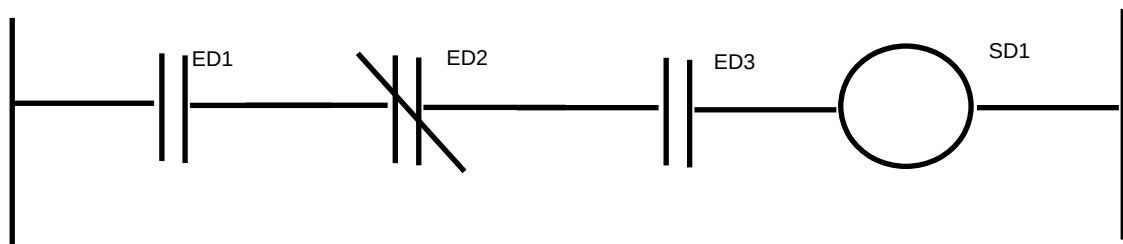


Figura 42 - Programa Ladder com lógica E.

A saída SD1 será acionada quando:

ED1 estiver acionada **E**

ED2 estiver não acionada **E**

ED3 estiver acionada.

Em álgebra booleana $SD1 = \overline{ED2} * ED1 * ED3$

Forma equivalente para escrita em texto estruturado seria:

```
IF ED1 AND NOT ED2 AND ED3 THEN
    SD1 = 1
ELSE
    SD1 = 0
END_IF
```

Figura 43 - Programa em texto estruturado com lógica E.

A lógica **OU** é conseguida com a associação paralela, acionando a saída desde que pelo menos um dos ramos paralelos estejam fechados

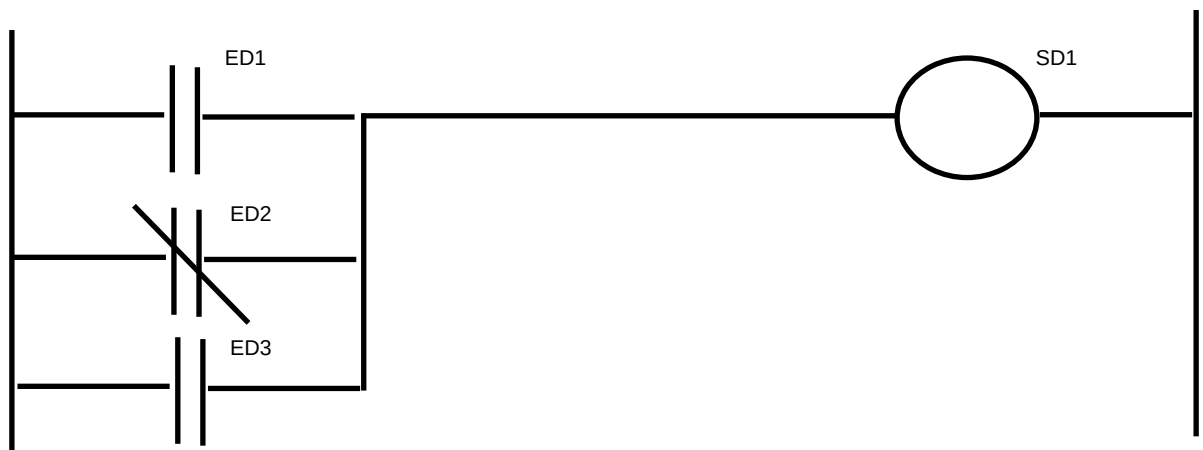


Figura 44 - Programa Ladder com lógica E.

A saída SD1 será acionada se:

ED1 for acionada **OU**

ED2 não for acionada **OU**

ED3 for acionada.

O que equivale a lógica booleana. $SD1 = ED1 + \overline{ED2} + ED3$

Forma equivalente para escrita em texto estruturado seria:

```
IF ED1 OR NOT ED2 OR ED3 THEN
    SD1 = 1
ELSE
    SD1 = 0
END_IF
```

Figura 45 - Programa em texto estruturado com lógica OU.

Com associações mistas criam-se condições mais complexas como a do exemplo a seguir

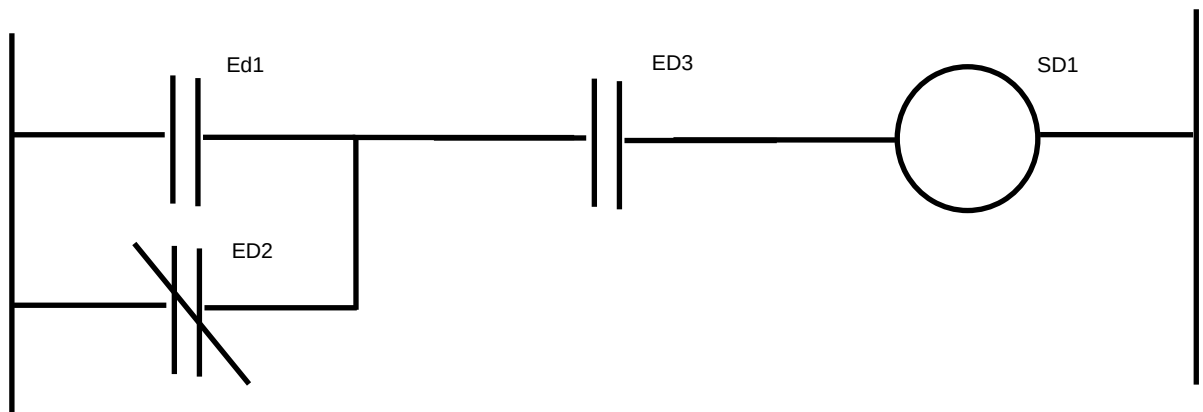


Figura 46 - Programa Ladder com lógica mista

Neste caso a saída é acionada quando:

ED3 for acionada **E**

ED1 for acionada **OU**

ED3 for acionada **E**

ED2 não for acionada.

Em lógica booleana $SD1 = ED3 * (ED1 + \overline{ED2})$

Forma equivalente para escrita em texto estruturado seria:

```
IF (ED1 OR NOT ED2) AND ED3 THEN
    SD1 = 1
ELSE
    SD1 = 0
END_IF
```

Figura 47 - Programa em texto estruturado com lógica E.

7.1.2 INSTRUÇÕES

Na CPU o programa residente possui diversos tipos de blocos de funções. Na listagem a seguir são apresentados alguns destes blocos:

- contador;
- temporização de energização;
- temporização de desenergização;
- adição de registros;
- multiplicação de registros;
- divisão de registros;
- extração de raiz quadrada;
- bloco OU lógico de duas tabelas;
- bloco E lógico de duas tabelas;
- ou exclusivo lógico de duas tabelas;
- deslocar bits através de uma tabela-direita;

- deslocar bits através de uma tabela-esquerda;
- mover tabela para nova localização;
- mover dados para memória EEPROM;
- mover inverso da tabela para nova localização;
- mover complemento para uma nova localização;
- mover valor absoluto para uma nova localização;
- comparar valor de dois registros;
- ir para outra sequência na memória;
- executar sub-rotina na memória;
- executar algoritmo PID;
- etc.

7.1.3 INSTRUÇÕES BÁSICAS

As instruções básicas são representadas por blocos funcionais introduzidos na linha de programação em lógica Ladder. Estes blocos funcionais podem se apresentar de formas diferentes de um CLP para outro, mas a filosofia de funcionamento é invariável. Estes blocos auxiliam ou complementam o controle do equipamento, introduzindo na lógica Ladder instruções como de temporização, contagem, soma, divisão, subtração, multiplicação, PID, conversão BCD/Decimal, conversão Decimal/BCD, raiz quadrada, etc.

O bloco funcional possui pontos de entrada (localizados à esquerda) e pontos de saída (localizados à direita do bloco), também possui campos de entrada de informações como: número do registro, memória, ponto de entrada analógico, bit de saída, bit de entrada, ponto de saída analógico, constantes, etc (figura seguinte).

As instruções seguintes serão explicadas supondo o byte de oito bits. A análise para o byte de dezesseis bits é exatamente a mesma.

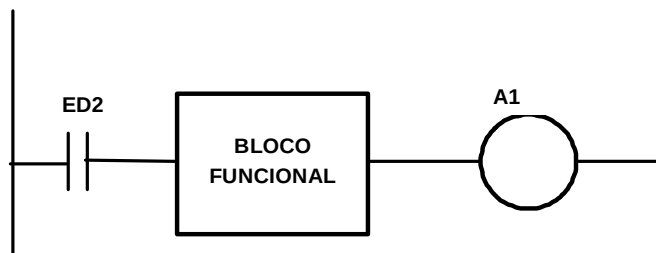


Figura 48 - Esquema do uso de um bloco funcional no programa Ladder.

7.1.3.1 INSTRUÇÃO DE TEMPORIZAÇÃO

O temporizador conta o intervalo de tempo transcorrido a partir da sua habilitação até este se igualar ao tempo preestabelecido. Quando a temporização estiver completa esta instrução eleva a nível 1 um bit próprio na memória de dados e aciona o operando a ela associado.

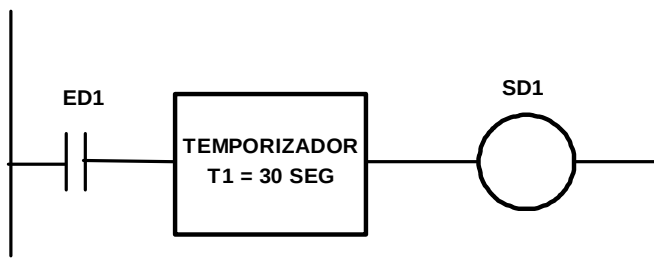


Figura 49 - Programa Ladder com bloco funcional de temporização.

Segundo exemplo, quando ED1 for acionada, o temporizador será habilitado e imediatamente após 30 segundos a saída SD1 será acionada. Quando ED1 for desacionada, o temporizador será desabilitado, ou desenergizado, desacionando a saída SD1. Em alguns casos, esta instrução apresenta duas entradas uma de habilitação da contagem e outra para zeramento ou reset da saída.

Para cada temporizador destina-se pelo menos um endereço de memória de dados onde o valor prefixado será armazenado, mas também são usados endereços para efetivar o funcionamento do temporizador.

Na memória de dados do CLP, o temporizador ocupa três bytes para o controle. O primeiro byte reservado para o dado prefixado, o segundo byte reservado para a temporização e o terceiro byte reservado para os bits de controle da instrução temporizador.

- 1º byte = valor prefixado de 30 seg.
- 2º byte = tempo transcorrido
- 3º byte = bits de controle D.E. (bit de entrada) e D.S. (bit de saída).

.....Os temporizadores podem ser TON (temporiza no acionamento) e TOFF (temporiza no desacionamento).

7.1.3.2 INSTRUÇÃO DE CONTAGEM

O contador conta o número de eventos que ocorre e deposita essa contagem em um byte reservado. Quando a contagem estiver completa, ou seja, igual ao valor prefixado, esta instrução energiza um bit de contagem completa. A instrução contador é utilizada para energizar ou desenergizar um dispositivo quando a contagem estiver completa.

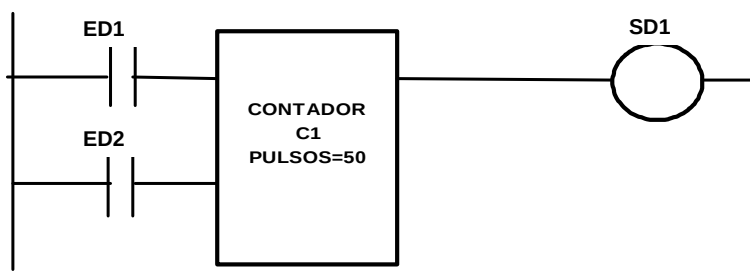


Figura 50 - Programa Ladder com bloco funcional de contagem.

Para cada contador destina-se pelo menos um endereço de memória de dados onde o valor prefixado será armazenado.

Na memória de dados do CLP, o contador ocupa três bytes para o controle. O primeiro byte reservado para o dado prefixado, o segundo byte reservado para a

contagem e o terceiro byte reservado para os bits de controle da instrução contador (figura seguinte).

- 1º byte = valor prefixado de 50
- 2º byte = contagem
- 3º byte = bits de controle D.E. (bit de entrada), D.S. (bit de saída) e D.R. (bit de reset).

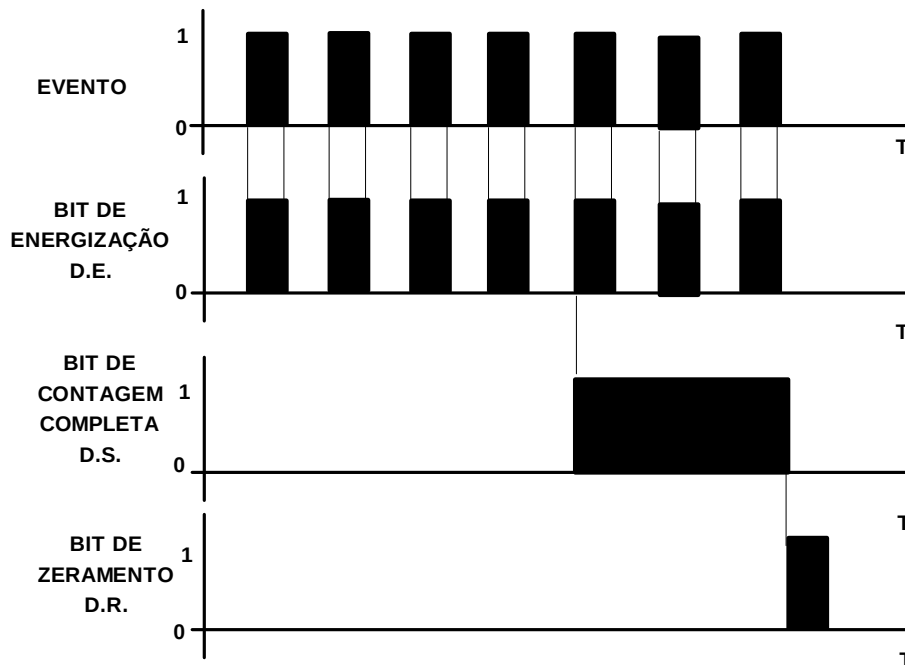


Figura 51 - Gráficos para demonstração do funcionamento do contador.

7.1.3.3 INSTRUÇÃO MOVER

A instrução mover transfere dados de um endereço de memória para outro endereço de memória, manipula dados de endereço para endereço, permitindo que o programa execute diferentes funções com o mesmo dado. Na verdade trata-se de um copy, pois o dado original é mantido.

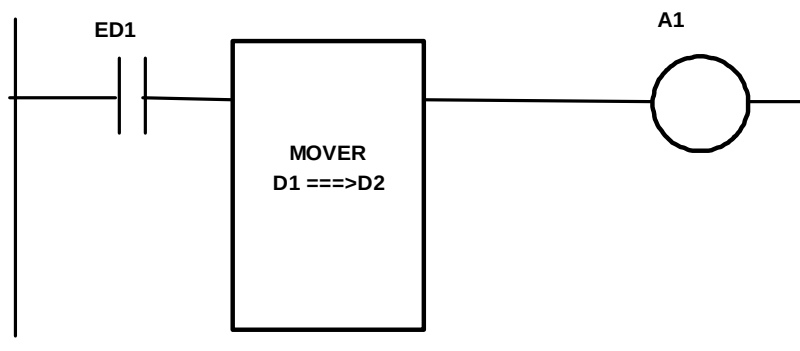


Figura 52 - Programa Ladder com bloco funcional para mover dado.

Abaixo tem-se dois endereços da memória de dados do CLP. Observe que o dado de D1 é distinto de D2.

	B7	B6	B5	B4	B3	B2	B1	B0
D1	0	0	0	0	1	1	1	1
D2	0	0	1	1	0	0	0	0

Supondo que a instrução mover tenha sido acionada e que a movimentação será de D1 para D2.

	B7	B6	B5	B4	B3	B2	B1	B0
D1	0	0	0	0	1	1	1	1
D2	0	0	0	0	1	1	1	1

Observe que o conteúdo de D2 foi alterado. No momento em que a instrução mover for desacionada, o dado de D2 permanecerá o mesmo.

Enquanto ED1 estiver acionada o dado será movido uma vez a cada ciclo de varredura, portanto ED1 deve ser acionado e desacionado rapidamente.

Tem-se a seguir o gráfico que ilustra antes e depois do acionamento de ED1 para a instrução mover.

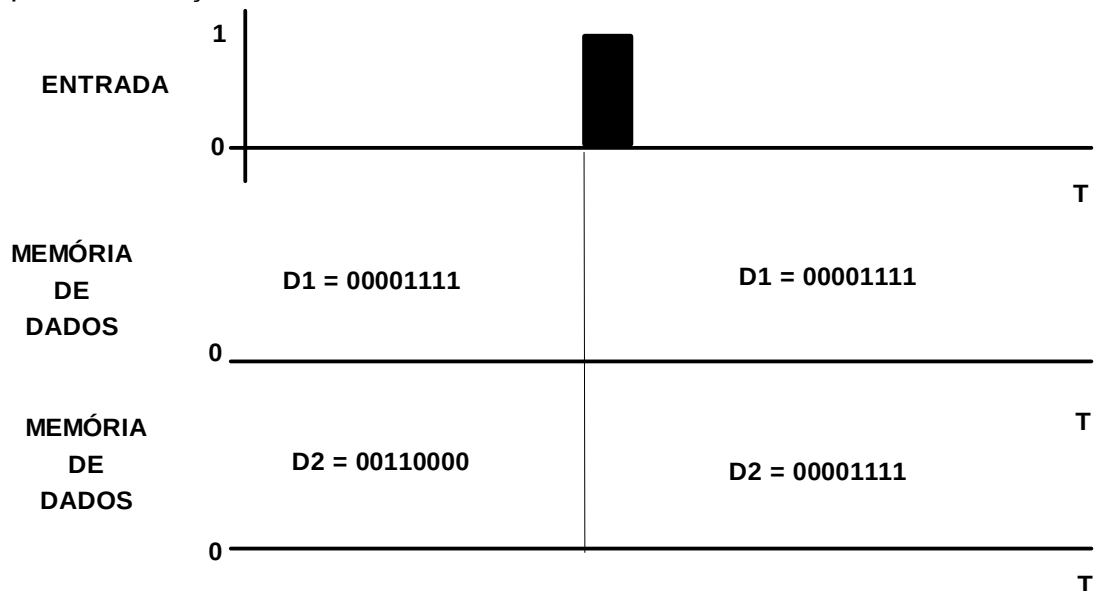


Figura 53 - Gráficos para demonstração do funcionamento do bloco mover.

7.1.3.4 INSTRUÇÃO COMPARAR

A instrução comparar verifica se o dado de um endereço é igual, maior, menor, maior/igual ou menor/igual que o dado de um outro endereço, permitindo que o programa execute diferentes funções baseadas em um dado de referência.

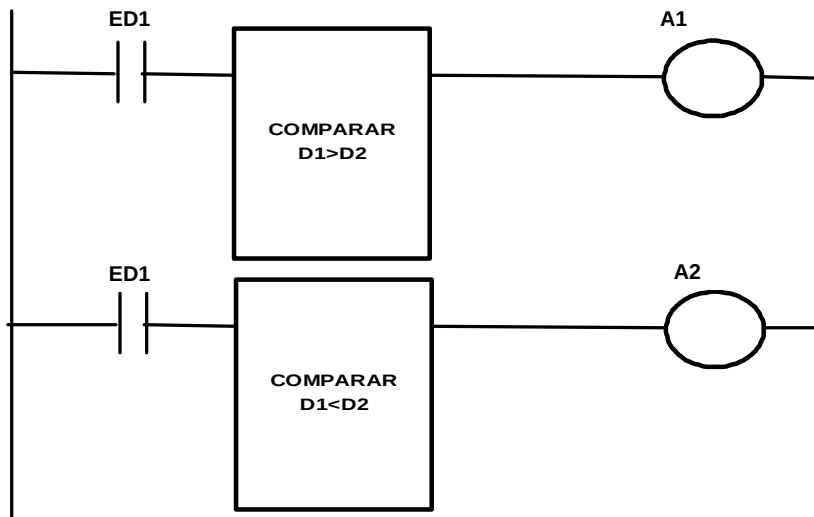


Figura 54 - Programa Ladder com blocos funcionais de comparação maior e menor.

No exemplo, quando a entrada ED1 for acionada as duas instruções de comparação serão acionadas, se D1 for maior que D2 o bit auxiliar A1 será acionado, se D1 for menor que D2 o bit auxiliar A2 será acionado. A comparação só existirá se a entrada ED1 estiver acionada, caso contrário os dois bits A1 e A2 serão desacionadas.

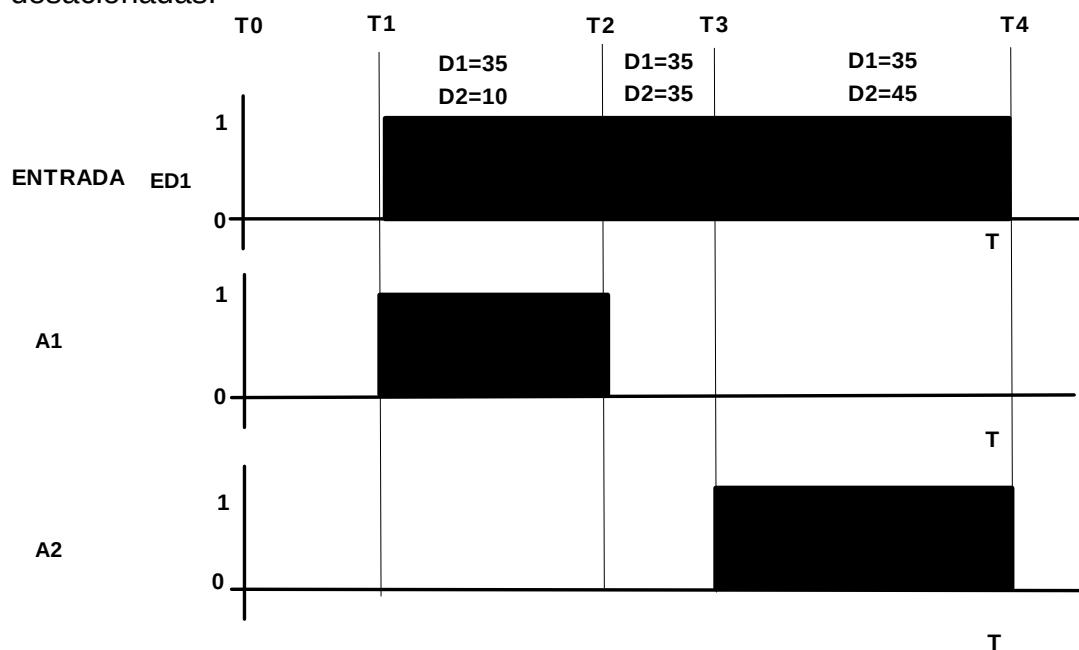


Figura 55 - Gráficos para demonstração do funcionamento do bloco de comparação maior e menor.

Observe o gráfico acima, entre T0 e T1 a entrada ED1 está desativada, logo não há comparação e os bits auxiliares A1 e A2 estão em nível lógico 0. Entre T1 e T2 o dado D1 se encontra com valor maior que D2, logo a instrução de comparação ativa o bit A1. Entre T2 e T3 o dado D1 é igual a D2, como não há instrução de igualdade as saídas estarão desativadas. Entre T3 e T4 o dado D1 é menor que D2, logo o bit A2 será ativado, a partir de T4 a entrada ED1 foi desacionada, portanto as comparações são desativadas e as saídas irão para estado lógico "0".

A mesma análise é válida para a instrução igual a, maior igual a e menor igual a.

7.1.3.5 INSTRUÇÕES MATEMÁTICAS

INSTRUÇÃO SOMA

Permite somar valores na memória quando habilitado. Nesta instrução podem-se usar os conteúdos de um contador, temporizador, byte da memória imagem, byte da memória de dados.

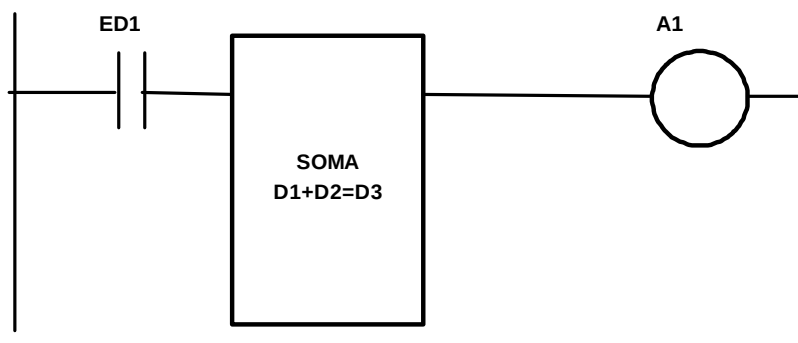


Figura 56 - Programa Ladder com bloco funcional matemático de adição.

Nesta instrução de programa, quando ED1 for acionada, a soma do dado 1 com o dado 2 será depositado no dado 3, portanto o conteúdo do dado 3 não deverá ter importância. Caso o conteúdo do dado 3 seja importante, o mesmo deve ser movido para um outro endereço ou o resultado da soma depositado em outro endereço.

Enquanto ED1 estiver acionado o dado D1 será somado com D2 e depositado no dado D3 a cada ciclo de varredura, portanto ED1 deve ser acionado e desacionado rapidamente.

Abaixo tem-se três endereços da memória de dados do CLP.

	B7	B6	B5	B4	B3	B2	B1	B0
D1	0	0	0	1	1	0	1	0
D2	0	0	0	0	1	1	1	1
D3	0	0	0	0	1	0	0	0

.....Supondo que a instrução somar tenha sido acionada e que a soma será de D1 e D2 em D3.

D1 equivale em decimal a 26 e D2 a 15, a soma resultará 41 no D3.

	B7	B6	B5	B4	B3	B2	B1	B0
D1	0	0	0	1	1	0	1	0
D2	0	0	0	0	1	1	1	1
D3	0	0	1	0	1	0	0	1

Observe que o conteúdo de D3 foi alterado, no momento em que a instrução soma for desacionada, os dados de D1 e D2 permanecerão os mesmos.

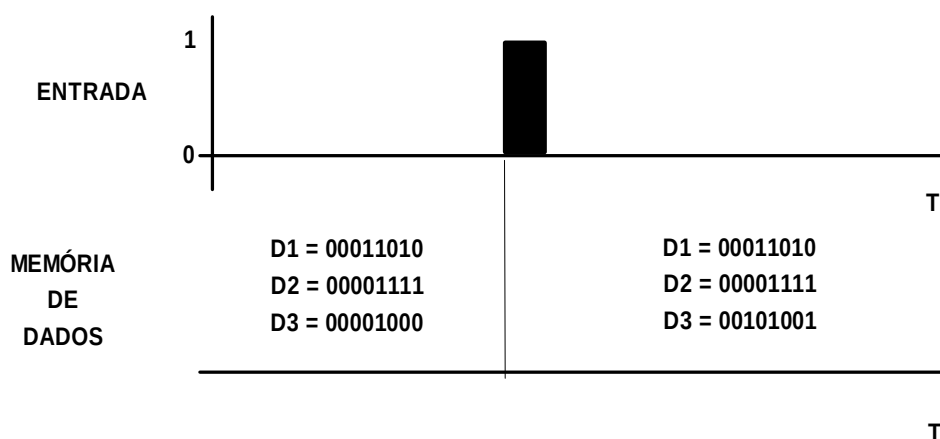


Figura 57 - Gráficos para demonstração do funcionamento do bloco soma.

O bit auxiliar A1 será acionada quando a soma for concluída.

Caso o resultado da soma não ultrapasse o limite máximo (overflow), o bit auxiliar A1 será acionada. Em alguns casos o um bit, do byte de controle da instrução soma, assume valor lógico “1”, determinando o estouro da capacidade. Através deste bit é possível de se determinar quando a soma ultrapassou ou não o valor máximo.

INSTRUÇÃO SUBTRAÇÃO

Permite subtrair valores na memória quando habilitado. Nesta instrução podem-se usar os conteúdos de um contador, temporizador, byte da memória imagem, byte da memória de dados.

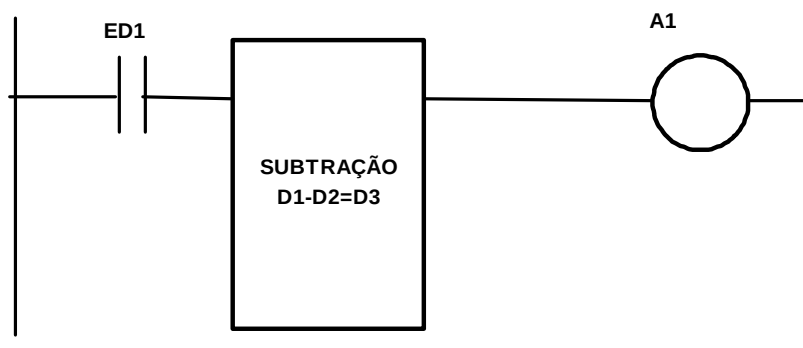


Figura 58 - Programa Ladder com bloco funcional matemático de subtração.

Nesta instrução de programa, quando ED1 for acionada, a subtração do dado 1 com o dado 2 será depositado no dado 3, portanto o conteúdo do dado 3 não deverá ter importância. Caso o conteúdo do dado 3 seja importante, o mesmo deve ser movido para um outro endereço ou o resultado da soma depositado em outro endereço.

Enquanto ED1 estiver acionado o dado D1 será subtraído do dado D2 e depositado no dado D3 a cada ciclo de varredura, portanto ED1 deve ser acionado e desacionado rapidamente.

Abaixo vêm-se três endereços da memória de dados do CLP.

	B7	B6	B5	B4	B3	B2	B1	B0
D1	0	0	0	1	1	0	1	0
D2	0	0	0	0	1	1	1	1
D3	0	0	0	0	0	0	0	0

Supondo que a instrução subtração tenha sido acionada e que a subtração será de D1 menos D2 em D3.

D1 equivale em decimal a 26 e D2 a 15, a subtração resultará 11 no D3.

	B7	B6	B5	B4	B3	B2	B1	B0
D1	0	0	0	1	1	0	1	0
D2	0	0	0	0	1	1	1	1
D3	0	0	0	0	1	0	1	1

Observe que o conteúdo de D3 foi alterado, no momento em que a instrução soma for desacionada, os dados de D1 e D2 permanecerão os mesmos.

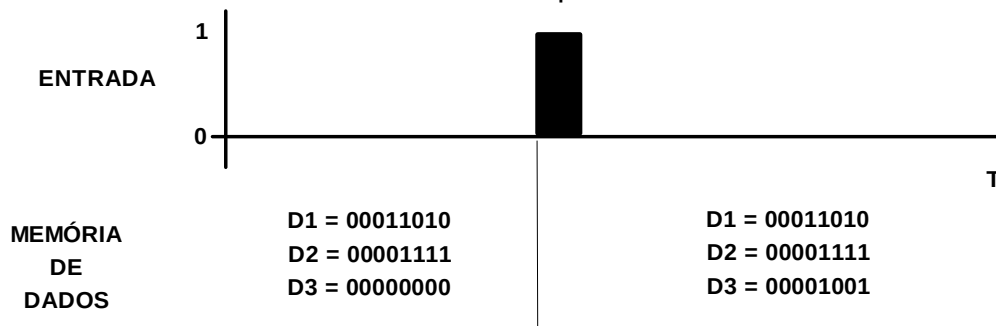


Figura 59 - Gráficos para demonstração do funcionamento do bloco de subtração.

Caso o resultado da subtração possua sinal negativo (underflow), o bit auxiliar será acionado. Em alguns casos o um bit, do byte de controle da instrução subtração, assume valor lógico "1". Através deste bit é possível de se determinar quando a subtração resultou positivo ou negativo.

INSTRUÇÃO MULTIPLICAÇÃO

Permite multiplicar valores na memória se a condição for verdadeira.

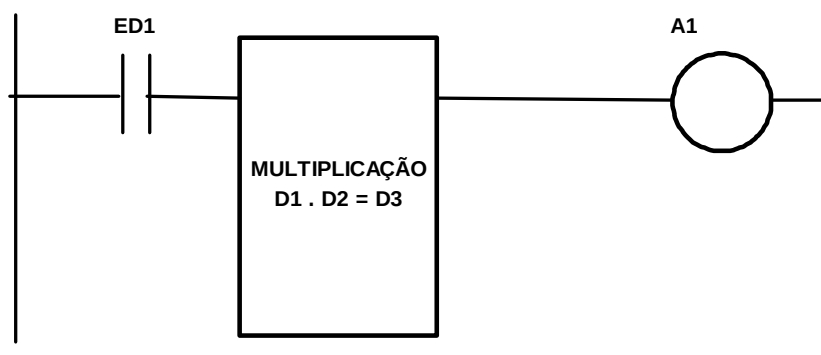


Figura 60 - Programa Ladder com bloco funcional matemático de multiplicação.

Observe os três endereços do mapa de memória apresentado.

	B7	B6	B5	B4	B3	B2	B1	B0
D1	0	0	0	1	1	0	1	0
D2	0	0	0	0	0	1	1	1
D3	0	0	0	0	0	0	0	0

Supondo que a instrução multiplicação tenha sido acionada por ED1 e que a multiplicação será de D1 por D2 em D3.

D1 equivale em decimal a 26 e D2 a 7, a multiplicação resultará 182 no D3.

	B7	B6	B5	B4	B3	B2	B1	B0
D1	0	0	0	1	1	0	1	0
D2	0	0	0	0	0	1	1	1
D3	1	0	1	1	0	1	1	0

Quando a entrada ED1 for acionada, a multiplicação do dado D1 pelo dado D2 será depositada no conteúdo do dado D3.

INSTRUÇÃO DIVISÃO

Permite dividir valores na memória quando habilitado.

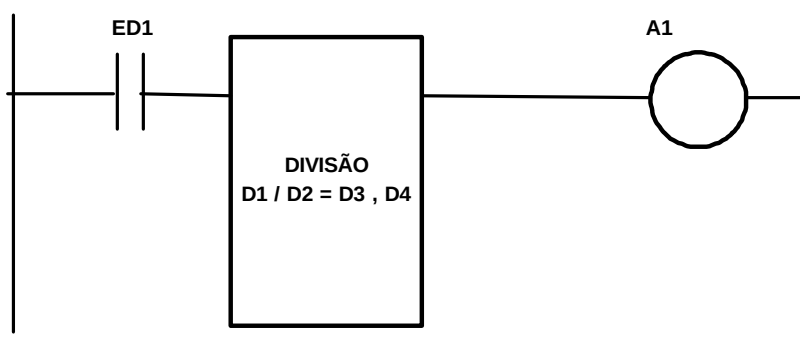


Figura 61 - Programa Ladder com bloco funcional matemático de divisão.

Observe os quatro endereços do mapa de memória apresentado.

	B7	B6	B5	B4	B3	B2	B1	B0
D1	0	0	1	1	0	0	1	0
D2	0	0	0	0	0	1	0	0
D3	0	0	0	0	0	0	0	0
D4	1	1	1	0	0	1	0	0

Supondo que a instrução divisão tenha sido acionada por ED1 e que a divisão será de D1 por D2 em D3, D4.

D1 equivale em decimal a 50 e D2 a 4, a divisão resultará 12,5 no D3, D4.

	B7	B6	B5	B4	B3	B3	B2	B1
D1	0	0	1	1	0	0	1	0
D2	0	0	0	0	0	1	0	0
D3	0	0	0	0	1	1	0	0
D4	0	0	0	0	0	1	0	1
D5	1	0	0	0	0	1	1	1

Quando a entrada ED1 for acionada, a divisão do dado D1 pelo dado D2 será depositada no conteúdo do dado D3, D4.

7.1.3.6 INSTRUÇÕES LÓGICAS

Estas instruções destinam-se à comparação lógica entre bytes. São recursos disponíveis para os programadores, podendo serem empregadas na análise de byte e diagnóstico de dados.

INSTRUÇÃO AND

Permite executar função AND com valores da memória quando habilitada .

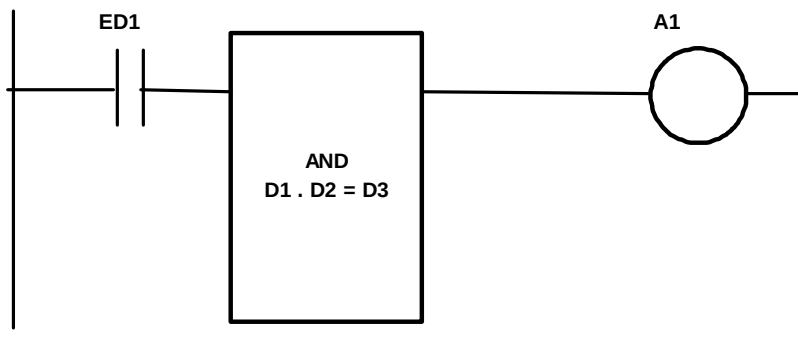


Figura 62 - Programa Ladder com bloco funcional lógica E.

Observe os três endereços do mapa de memória apresentado.

	B7	B6	B5	B4	B3	B2	B1	B0
D1	0	1	0	1	1	0	1	0
D2	0	1	0	0	0	1	1	1
D3	0	0	0	0	0	0	0	0

Supondo que a instrução AND tenha sido acionada por ED1 e que a instrução será de D1 and D2 em D3.

Observe a tabela verdade abaixo e verifique o resultado da análise AND entre os dois bytes D1 e D2.

A	B	S
0	0	0
0	1	0
1	0	0
1	1	1

A e B são as entradas e S é o resultado da operação E.

	B7	B6	B5	B4	B3	B2	B1	B0
D1	0	1	0	1	1	0	1	0
D2	0	1	0	0	0	1	1	1
D3	0	1	0	0	0	0	1	0

Quando a entrada ED1 for acionada, a instrução do dado D1 and dado D2 será depositada no conteúdo do dado D3.

INSTRUÇÃO OR

Permite executar função OU com valores da memória quando habilitada.

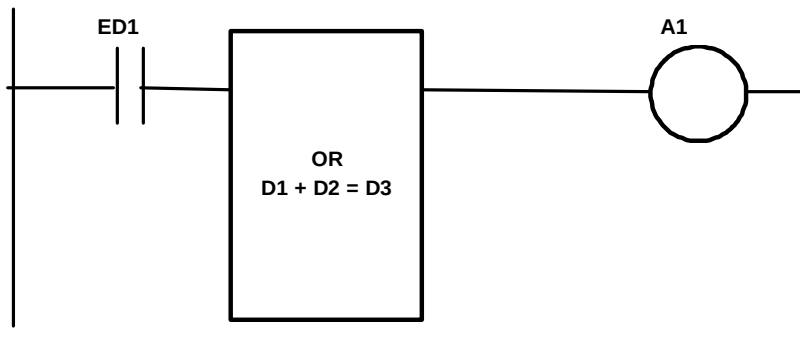


Figura 63 - Programa Ladder com bloco funcional lógica OU.

Observe os três endereços do mapa de memória apresentado.

	B7	B6	B5	B4	B3	B2	B1	B0
D1	0	1	0	1	1	0	1	0
D2	0	1	0	0	0	1	1	1
D3	0	0	0	0	0	0	0	0

Supondo que a instrução OR tenha sido acionada por ED1 e que a instrução será de D1 or D2 em D3.

Observe a tabela verdade abaixo e verifique o resultado da análise OR entre os dois bytes D1 e D2.

A	B	S
0	0	0
0	1	1
1	0	1
1	1	1

A e B são as entradas e S é o resultado da operação lógica OU inclusivo.

	B7	B6	B5	B4	B3	B2	B1	B0
D1	0	1	0	1	1	0	1	0
D2	0	1	0	0	0	1	1	1
D3	0	1	0	1	1	1	1	1

Quando a entrada ED1 for acionada, a instrução do dado D1 or dado D2 será depositada no conteúdo do dado D3.

INSTRUÇÃO XOR

Permite executar função ou exclusivo com valores da memória quando habilitada.

A1

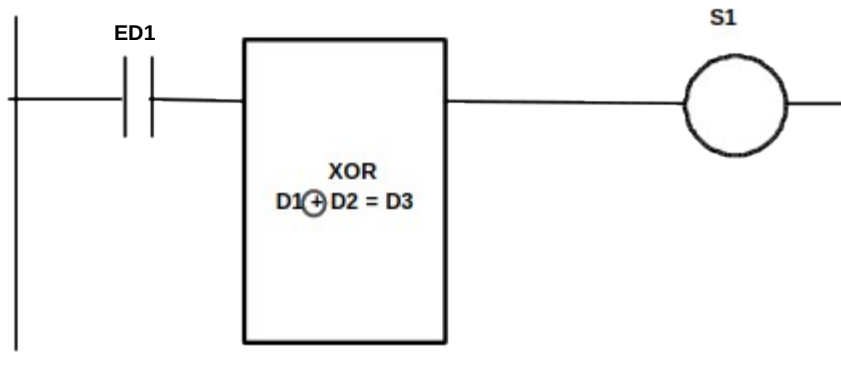


Figura 64 - Programa Ladder com bloco funcional lógica ou exclusivo.

Observe os três endereços do mapa de memória apresentado.

	B7	B6	B5	B4	B3	B2	B1	B0
D1	0	1	0	1	1	0	1	0
D2	0	1	0	0	0	1	1	1
D3	0	0	0	0	0	0	0	0

Supondo que a instrução XOR (ou exclusivo) tenha sido acionada por ED1 e que a instrução será de D1 xor D2 em D3.

Observe a tabela verdade abaixo e verifique o resultado da análise xor entre os dois bytes D1 e D2.

A	B	S
0	0	0
0	1	1
1	0	1
1	1	0

A e B são as entradas e S é o resultado da operação OU exclusivo.

	B7	B6	B5	B4	B3	B2	B1	B0
D1	0	1	0	1	1	0	1	0
D2	0	1	0	0	0	1	1	1
D3	0	0	0	1	1	1	0	1
D4	1	1	1	0	0	1	0	0
D5	1	0	0	0	0	1	1	1

Quando a entrada ED1 for acionada, a instrução do dado D1 XOR dado D2 será depositada no conteúdo do dado D3.

Obviamente estas são apenas algumas instruções que a programação Ladder dispõe. Uma série de outros recursos são disponíveis em função da capacidade do CLP em questão.

As instruções apresentadas servirão como base para o entendimento das instruções de programação Ladder de qualquer CLP, para tal conte e não dispense o auxílio do manual ou help on-line quando disponível no software de programação.

A utilização do software de programação é uma questão de estudo e pesquisa, uma vez que o layout de tela e comandos não são padronizados.

8 NOÇÕES DE SISTEMA SCADA COM USO DO CLP

Na indústria tem-se a necessidade de centralizar as informações de forma a termos o máximo dessas no menor tempo possível. Embora a utilização de painéis centralizados venha a cobrir esta necessidade, muitas vezes a sala de controle possui grandes extensões com centenas ou milhares de instrumentos tornando o trabalho do operador uma verdadeira maratona.

Os sistemas SCADA (*Supervisory Control and Data Acquisition*) são os sistemas de supervisão de processos industriais que coletam dados do processo através de remotas industriais, principalmente Controladores Lógico Programáveis (CLP), formatam estes dados, e os apresenta ao operador em uma multiplicidade de formas. O objetivo principal dos sistemas SCADA é propiciar uma interface de alto nível do operador com o processo informando-o "em tempo real" de todos os eventos de importância da planta.

O software de supervisão e controle (parte integrante do sistema SCADA) recebe as informações dos controladores concentrando todos os eventos ocorridos. Permite que um operador visualize imediatamente o que está acontecendo em cada processo. Isto faz com que seja possível alterar os parâmetros de controle de acordo com a necessidade. Além disso, o software de supervisão e controle permite armazenar todas as informações recebidas possibilitando ao usuário a análise dos acontecimentos com:

- Correção de desvios;
- Otimização do processo;
- Documentação de Partida/Lotes.

Isto significa:

- Mais segurança operacional;
- Melhor qualidade;
- Menor custo operacional.

O operador supervisiona e controla todo o processo por meio de um conjunto de telas que, dentro de um padrão, serão detalhadas de forma específica para cada processo e indústria. Além disso,o sistema supervisório veio para reduzir a dimensão dos painéis e melhorar o performance homem/máquina.

Estes software de supervisão e controle podem operar com qualquer equipamento de controle ou aquisição de dados, como por exemplo:

- Controladores multiloop;
- Controladores singleloop;
- Redes Fieldbus;
- Controladores Programáveis;
- Placas de aquisição de dados (DDC);
- Medidores de vazão;
- Entre outros.

A premissa básica para que exista esta interoperação é a comunicação que deve ser realizada por um driver ou servidor de comunicação (conjunto de drivers). O driver é um software responsável pela comunicação, nele está codificado o protocolo de comunicação do equipamento. A figura a seguir ilustra o esquema de um sistema SCADA básico.

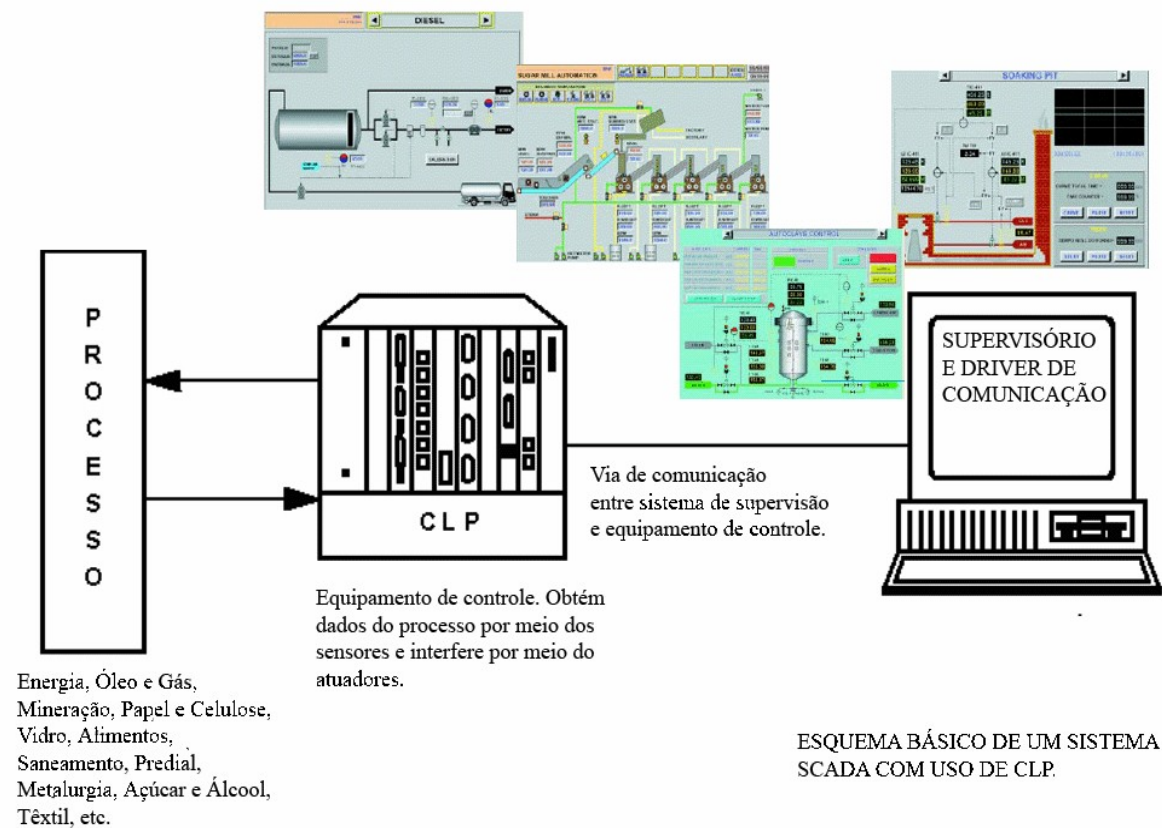


Figura 65 - Esquema básico de um sistema SCADA.

8.1 ARQUITETURA DA REDE CLP PARA SISTEMAS SCADA

Considerando a localização dos módulos de entrada e saída e rede de comunicação entre o CLP e a estação de programação. A arquitetura de rede do CLP pode ser classificada em:

- Local;
- I/O distribuído ou remotos;
- Rede de CLP's.

As figuras seguintes ilustram estas três arquiteturas de rede de CLP com sistema de supervisão.

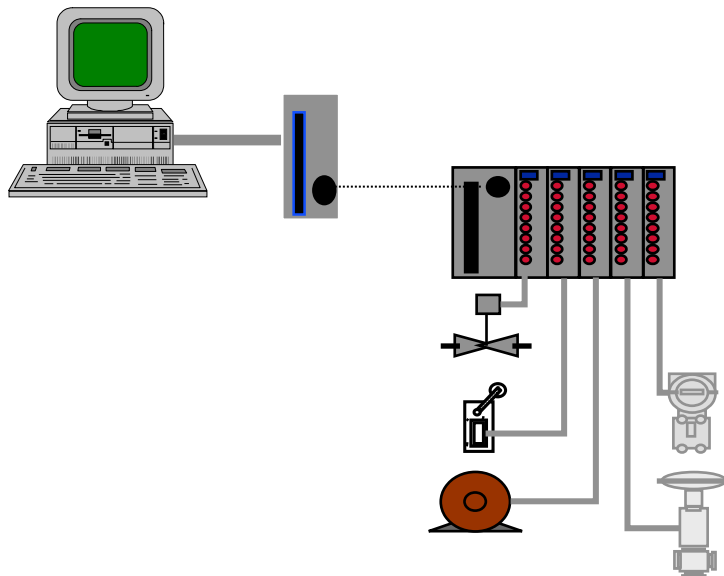


Figura 66 – Arquitetura local de rede CLP com uso do CLP modular ou compacto.

Nesse tipo de arquitetura, os módulos de I/O montados localmente em um CLP modular e a comunicação é do tipo ponto-a-ponto.

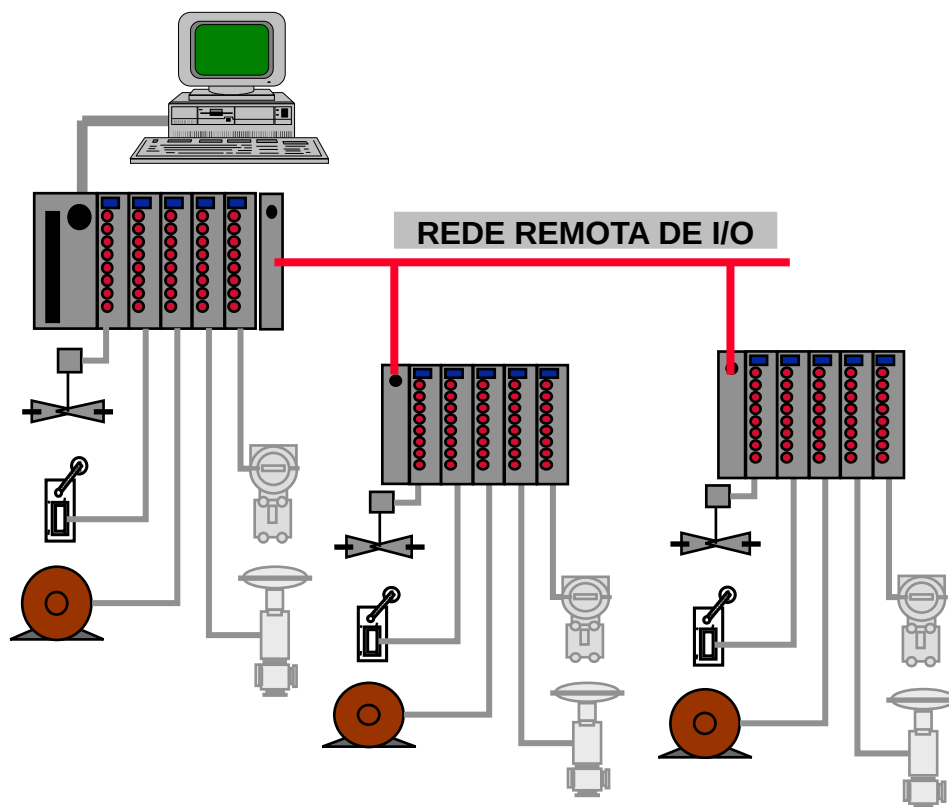


Figura 67 - Arquitetura local de CLP com I/O remotos ou distribuídos.

A instalação de um sistema automático com o uso de I/O locais, requer um gasto considerável de cabeamento, borneiras, caixas de passagem, bandejas, projeto e mão-de-obra para a instalação. Os blocos I/O remotos possibilitam uma redução drástica destes gastos, uma vez que todos os sinais não serão encaminhados para o rack do CLP e sim para pontos de entradas e saídas que ficarão localizados no campo.

Estes módulos de I/O, também conhecidos como “remotas” de I/O, são independentes e configuráveis. Interligados entre si através de um barramento de campo (fieldbus) proprietário ou de padrão aberto. Nesta arquitetura existe a necessidade de cartões de interface para conexão entre os rack's remotos e o rack central.

Um barramento permite aprimorar o controle de I/O através do uso de comandos de comunicação no programa. O barramento também pode ser usado inteiramente para o controle de I/O, com múltiplos dispositivos de I/O e sem comunicação adicional. Pode ainda ser dedicado à comunicação da CPU, com múltiplas CPUs e sem dispositivos de I/O. Sistemas mais complexos também podem ser desenvolvidos, com CPUs duplas e uma ou mais CPUs adicionais para a monitoração de dados

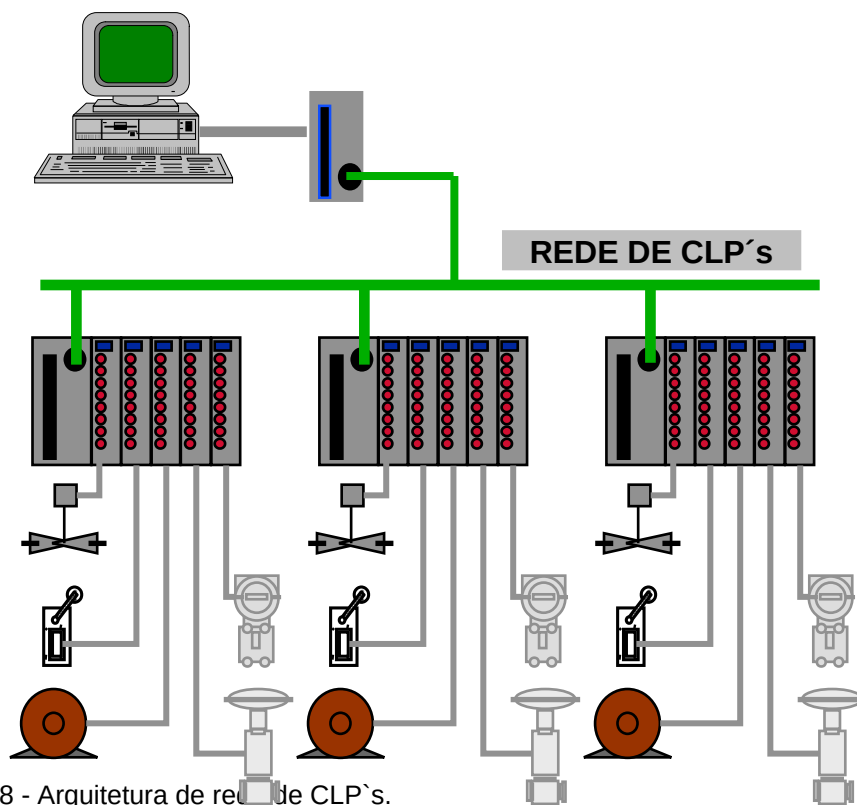


Figura 68 - Arquitetura de rede de CLP's.

Módulos de I/O montados localmente. Normalmente, a comunicação dos CLP's com o sistema de supervisão é do tipo mestre-escravo ou polling.

9 CRITÉRIOS PARA AQUISIÇÃO DE UM CLP

9.1 CRITÉRIOS DE CLASSIFICAÇÃO

Existem vários critérios de classificação de controladores programáveis. A classificação adotada pelo mercado americano é em função da quantidade de E/S e pelo porte físico de cada equipamento.

- Micro (até 64 E/S);
- Pequeno (até 256 E/S);
- Médio (até 1.024 E/S);
- Grande (até 4.096 E/S);

- Muito Grande (acima de 4.096 E/S).

9.2 CRITÉRIOS DE AVALIAÇÃO PARA ESPECIFICAÇÃO E COMPRA DE UM CP

Procedimentos para especificação:

→ Definir uma configuração mínima do CP, que atenda a todas as necessidades de operação e de processo.

Exemplo:

Se uma CPU simples basta ou é necessária uma configuração redundante (hot-standby) e troca de cartões com o sistema energizado (hot-swap).

Mesmo tendo absoluta certeza do número de E/S necessários, deve-se acrescentar pelo menos uma folga de 10% com relação a configuração prevista.

→ *Analisar alternativas:*

Caso o CP se enquadre na classificação de grande porte, optar por uma arquitetura distribuída, isto é, analisar se a utilização de um sistema de CP's de menor porte em rede não resolveriam melhor esta aplicação.

A existência de alternativas viáveis tecnicamente pode levar a uma solução econômica mais interessante.

→ *Definir quais os índices mínimos de desempenho que se deseja do sistema.*

Levar em conta o scan total do CP (processamento + atualização de E/S), mais os retardos entre os sinais lógicos de E/S e os sinais físicos, isto é, o tempo entre o CP compreender um sinal físico de entrada e o sinal lógico de saída ser convertido em uma ação de campo.

→ *Dimensionar a memória necessária para a execução adequada da tarefa a qual o CP se destina.*

A memória deve ser suficiente para armazenar o programa do usuário, os dados, operandos auxiliares, valores de contagem, temporização, etc.

→ *Verificar a necessidade do processo exigir funções especiais na programação:*

- Módulos Inteligentes (PID, etc);
- Comunicação via rede;
- Rotinas com execução periódicas;
- Rotinas com frequência de execução diferente do ciclo normal do CP;
- Procedimentos de emergência em caso de alarme;
- etc.

☐☐☐ ANÁLISE DO FORNECEDOR

→ *Verificar se o fornecedor possui CP's instalados em processos similares ao em estudo;*

→ *Verificar se o fornecedor tem uma equipe capacitada para suporte pré e pós venda;*

→ Verificar se o fornecedor tem condições de manter a continuidade do produto;

☐☐ ASPECTOS TÉCNICOS DO PRODUTO

→ Evitar aceitar equipamentos com menos de um ano no mercado e com pelo menos uma instalação industrial operando por este período;

→ O equipamento deve ter modularidade, isto é, capacidade de ampliação;

→ Deve suportar rede de comunicação, mesmo que não se utilize de imediato, prevendo integração com outras áreas da empresa;

→ Deve empregar tecnologia mais avançada disponível no momento da compra;

☐☐ ASPECTOS CONTRATUAIS

→ Devem ser acordadas visitas de inspeção e procedimentos de teste elétrico e funcional do equipamento fornecido;

→ Exigir suporte ao equipamento por pelo menos a vida útil da instalação;

→ Caso a data de entrega seja um ponto crítico no cronograma da obra, defini-la e inserir uma cláusula de multa por atraso;

→ Cuidado na alteração da arquitetura após a colocação do pedido, pois alguns adendos podem custar mais caros que o equipamento que originalmente seria comprado.

Após toda a análise de mercado dois ou mais fornecedores atenderem às especificações em todos os aspectos, o último critério a ser empregado deve ser o preço.

10 BIBLIOGRAFIA RECOMENDADA

- VIANNA, W. S. Apostila de Controlador Lógico Programável – Centro Federal de educação Tecnológica de Campos. 1995.
- Documentação Técnica da norma IEC 61131. Dados obtidos em 06 de abril de 2008 por meio da [url:http://www.iec61131.com.br](http://www.iec61131.com.br)
- MORAIES, C. M., CASTRUCCI, P. L. Engenharia de Automação Industrial. 2ª. Ed. LTC. 2007.
- OLIVEIRA, Júlio César P. Controlador Programável, São Paulo: Makron Books, 1993.
- NATALE, Ferdinando. Automação Industrial, São Paulo: Editora Érica, 1992.
- OIKO, RENATO K./SARROUF, LUIZ P. “Controladores Programáveis-como comprar”. Instec, São Paulo, nº52, pp 32-45, março, 1992.